Matrix Theory, Spectral Drift, and Bellman-Optimal Control of Fault Diagnosis in Cascaded Partially-Observable Transducers

With Step-by-Step Worked Derivations, $N = 16$ Chained SSH Bastion Verification, and LLM Failure Analysis

Formal Systems Verification & Network Diagnosis Study

October 25, 2026

Abstract

Diagnosing multi-hop networked protocols such as OpenSSH (`ProxyJump`) across N chained servers involves interacting with a sequential cascade of $D = N \cdot M$ heterogeneous state transducers spanning local IPC sockets (`ssh-agent`), interactive TTY gates (`/dev/tty`), UDP/TCP transport layers, cryptographic handshakes, and remote daemon policies (`sshd`). Standard end-to-end execution induces severe *causal censoring* and *observational aliasing*: a 2^D-dimensional latent state space collapses through a rank-deficient observation matrix into a handful of ambiguous terminal strings (`Permission denied`, `Operation timed out`). In this monograph, we formulate cascaded fault diagnosis strictly through **Matrix Theory**, **Partially Observable Markov Decision Processes (POMDPs)**, and **Information Theory**. We derive the Nilpotent Shift-Transition Algebra of sequential cascades, characterize observational aliasing via the nullspace and Kronecker structure of the column-stochastic observation matrix $\mathbf{O}_a$, and prove the *ProxyJump Non-Propagation Paradox* in OpenSSH's `ssh.c` option-inheritance operator. We provide complete step-by-step matrix derivations for both a **Simple Worked Example** ($D = 7, W = 8$) and a **Complex 16-Hop Production Instance** ($D = 128, W = 129, K = 5$ simultaneous masked faults), illustrated with vector-rendered diagrams. Finally, we prove via Foster–Lyapunov spectral drift analysis why unbisected Large Language Model (LLM) agents suffer an exponential hitting-time blowup $\mathbb{E}[\tau_0] = \Omega((\alpha M_{\text{alias}})^{M_{\text{alias}}-1})$ under collateral intervention noise $\alpha > 0$, whereas our exact C99 Dynamic Programming Prefix-Bisection algorithm achieves the Shannon lower bound $V^*(D+1) \approx \log_2(D+1)$ with zero erroneous interventions.

Contents

1 Introduction: The Cascaded SSH Diagnosis Problem **4**

2 Matrix Theory of Cascaded State Transducers **4**
2.1 The Nilpotent Shift-Transition Algebra of a Cascade 4
2.2 The Censoring Surjection Matrix $\mathbf{C} \in \{0,1\}^{(D+1)\times 2^D}$ 5

3 Observation Matrices, Rank Deficiency, and Prefix Unimodularity **5**
3.1 The End-to-End Kronecker Aliasing Matrix $\mathbf{O}_{\text{e2e}}$ 5
3.2 Bayesian Belief Updating and the Unimodular Prefix Matrix $\mathbf{\Pi}$ 6

4 Step-by-Step Worked Examples of Optimal Matrix Bisection **7**
4.1 Worked Example 1 (Simple): $N = 2$ Hops, $D = 7$ Stages ($W = 8$ States) with Masked Faults 7
4.2 Worked Example 2 (Complex): $N = 16$ Chained SSH Servers ($D = 128$, $K = 5$ Masked Faults) 8

5 Formal Matrix Proof of the OpenSSH `ProxyJump` Paradox **10**

6 Markov Chain Spectral & Drift Analysis of Unbisected LLM Agents **11**

7 Computational Complexity Theory: NP-Completeness vs. Total Unimodularity **11**
7.1 NP-Completeness of General Non-Cascaded Fault Diagnosis 12
7.2 Why the Sequential Cascade Collapses from NP-Hard to P: Total Unimodularity (TUM) 12

8 Linear Programming (LP), MILP, and Convex Programming Formulations **13**
8.1 Exact Linear Programming (LP) Formulation of the Bellman Equation 13
8.2 Exact LP Fault Localization via Totally Unimodular First-Differences 14
8.3 Mixed-Integer Linear Programming (MILP) for General Multi-Fault & Set-Cover Diagnosis 14
8.4 Convex Programming (CP) for Maximum Information Gain & Randomized Probe Design 15

9 Can Matrix-Multiplication LLMs Solve This Optimally? Circuit Expressivity vs. Meta-Cognitive Solver Routing **16**
9.1 Constructive Proof: A 2-Layer Transformer Matrix Circuit Implements Exact Bellman Bisection 16
9.2 Why Standard Autoregressive LLMs Fail Despite Circuit Capacity 17
9.3 Meta-Cognitive Detection and the Epistemic Switching / Disclosure Protocol . . . 18

10 Isomorphic 256×256 2D Grid Game & Abramsky–Hyland–Ong Game Semantics **19**
10.1 Geometric Construction of the 256×256 2D Dungeon Board 19
10.2 Abramsky–Hyland–Ong Game Semantics & Full Abstraction Proof 20

11 Strict C99 Implementation of the Optimal Bellman Solver **21**

12 Concrete Operational Correspondence: 2D Game Mechanics $\leftrightarrow$ Real-Life OpenSSH Commands, System States, and Production Consequences **22**
12.1 Gadget-to-OpenSSH Protocol Substage Correspondence ($m \in \{0,\dots,7\}$) 22
12.2 Action-to-Command Correspondence and Concrete Worked Examples 23

13 Joint Symptom-Subspace Bisection Theory: Combining $\mathbf{O}_{\mathbf{e2e}}^{-1}(o)$ with Π **25**
13.1 Kronecker Symptom Fiber Decomposition and Submatrix Total Unimodularity . . 25

14 Empirical Benchmark of Frontier Coding Agents (`Codex / gpt-6-astra ultra`) and Physical Compute, FLOPs, & Cost Scaling **27**
14.1 The Five Controlled Agentic Experiments 28
14.2 First-Principles Derivation of Silicon Cycles, PetaFLOPs, Energy, and Monetary Cost 31

15 Production Economics & Infrastructure Risk Analysis: Low-Setting vs. Ultra-Setting Codex LLMs vs. Specialized Optimal C99 Solver **32**
15.1 Token & Probe Economics: Lowest-Setting Codex vs. Highest-Setting (`gpt-6-astra ultra`) vs. Specialized C99 33
15.2 Concrete Worst-Case Production Server Outage Analysis: What Happens When an Agent Executes `repair 22` and `repair 26` (`Run 0a`) or Loops Under $\alpha = 0.25$ . . 34

16 How Can One Problem Have Three Distinct "Optimal" Solutions? The Hierarchy of Oracle Coupling Regimes & Order-Statistic Pass-Chain DP **35**
16.1 The Four Nested Oracle Regimes ($\mathfrak{O}_0 \supset \mathfrak{O}_{\mathrm{III}} \supset \mathfrak{O}_{\mathrm{II}} \supset \mathfrak{O}_{\mathrm{I}}$) 36

16.2 Theorem 15.1: Exact Coupled Pass-Chain Dynamic Programming ($619/129 \approx 4.79845$) and the Rainbow-Interval Property 37
16.3 Theorem 15.2: Order-Statistic Multi-Fault Pass-Chain DP ($w_r(k) = \binom{127-k}{r-1}$) and Claude's 21-Probe Solve 38

17 Foundational Bibliography **40**

1 Introduction: The Cascaded SSH Diagnosis Problem

Consider an enterprise or industrial Operational Technology (OT) network comprising N sequentially chained SSH jump hosts $H_0, H_1, \ldots, H_{N-1}$ (for instance, $N = 16$ bastions traversing corporate VPNs, cloud transit gateways, MPLS edges, and Purdue-level OT firewalls down to a field controller PLC). Each hop $i \in \{0, \ldots, N-1\}$ requires the successful execution of M sequential internal sub-stages (e.g., $M = 8$):

1. $m = 0$ (`SUB_AGENT_SOCK`): Local or forwarded `SSH_AUTH_SOCK` UNIX domain socket reachability.
2. $m = 1$ (`SUB_KEY_LOADED`): Private key decrypted in agent RAM or unlocked interactively via `/dev/tty`.
3. $m = 2$ (`SUB_DNS_UDP`): Forward hostname resolution over UDP port 53.
4. $m = 3$ (`SUB_TCP_ROUTE`): TCP 3-way handshake (`SYN`, `SYN-ACK`, `ACK`) and MTU path integrity.
5. $m = 4$ (`SUB_HOSTKEY_KEX`): Cryptographic key exchange (KEX) and `known_hosts` verification.
6. $m = 5$ (`SUB_REMOTE_SSHD`): Remote `sshd` daemon pre-auth checks (including server-side reverse DNS `UseDNS yes` `getnameinfo()` lookups over UDP/53).
7. $m = 6$ (`SUB_AUTH_KEYS`): Remote `StrictModes` directory ownership, `authorized_keys`, or PAM verification.
8. $m = 7$ (`SUB_NETCAT_FWD`): `SSH2_MSG_CHANNEL_OPEN` (`direct-tcpip` / `-W [%h]:%p`) forwarding permission (`AllowTcpForwarding`).

Let $D = N \cdot M$ denote the total number of sequential causal stages (for $N = 16, M = 8$, $D = 128$). A global latent fault configuration is a binary vector in the D-dimensional hypercube:

$$\boldsymbol{s} = (s_0, s_1, \ldots, s_{D-1})^\top \in \{0,1\}^D, \qquad |\mathcal{S}| = 2^D = 2^{128} \approx 3.4028 \times 10^{38}, \tag{1}$$

where $s_k = 1$ indicates stage $k = iM + m$ is operational, and $s_k = 0$ indicates stage k is faulty.

When an operator or LLM agent invokes an end-to-end connection attempt, two fundamental structural phenomena occur:

1. **Causal Censoring (Downstream Masking):** Because the protocol executes sequentially from stage 0 to $D-1$, execution halts deterministically at the *first* faulty stage $k^* = \min\{k : s_k = 0\}$. Every downstream fault $k > k^*$ is physically unreachable and exerts zero influence on the execution trace.

2. **Observational Aliasing (Many-to-One Projection):** The rich internal state trajectory of the D-stage cascade is compressed into a tiny alphabet of terminal observations $\mathcal{O}_{\text{e2e}}$ (e.g., `Permission denied (publickey)` or `Operation timed out`). A local `/dev/tty` passphrase block at $k = 1$, an intermediate firewall TCP `SYN` drop at $k = 59$, and a remote `sshd` reverse-DNS UDP/53 hang at $k = 93$ all map to the exact same timeout observation.

2 Matrix Theory of Cascaded State Transducers

2.1 The Nilpotent Shift-Transition Algebra of a Cascade

To represent the execution of a D-stage sequential cascade as a linear operator, we define the active execution state space of a single probe trace as a $(D+1)$-dimensional vector space with standard basis $\{\boldsymbol{e}_0, \boldsymbol{e}_1, \ldots, \boldsymbol{e}_D\}$ (where $\boldsymbol{e}_k$ represents the execution cursor arriving at stage k, and $\boldsymbol{e}_D$ represents End-to-End Success), augmented by D absorbing halting receptors $\{\boldsymbol{f}_0, \boldsymbol{f}_1, \ldots, \boldsymbol{f}_{D-1}\}$ recording the exact stage where execution aborted.

Let $\mathbf{J} \in \{0,1\}^{(D+1)\times(D+1)}$ denote the nilpotent lower-shift matrix on the active execution coordinates $\{\boldsymbol{e}_0, \ldots, \boldsymbol{e}_D\}$:

$$\mathbf{J} = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & 0 \end{pmatrix}, \qquad \mathbf{J}^{D+1} = \mathbf{0}, \tag{2}$$

and let $\mathbf{D}(\boldsymbol{s}) = \mathrm{diag}(s_0, s_1, \ldots, s_{D-1}, 0) \in \{0,1\}^{(D+1)\times(D+1)}$ be the diagonal stage pass-through matrix.

Theorem 2.1 (Block-Matrix Cascade Propagator and Censoring Operator)**.** *Let the composite single-step transition matrix on the augmented state space $\mathbb{R}^{(D+1)+D}$ (ordered as active states $\boldsymbol{e}_{0:D}$ followed by absorbing halted states $\boldsymbol{f}_{0:D-1}$) be written in block-matrix form:*

$$\mathbf{T}(\boldsymbol{s}) = \begin{pmatrix} \mathbf{J}\mathbf{D}(\boldsymbol{s}) + \boldsymbol{e}_D \boldsymbol{e}_D^\top & \mathbf{0}_{(D+1)\times D} \\ \mathbf{I}_{D\times(D+1)} - \widetilde{\mathbf{D}}(\boldsymbol{s}) & \mathbf{I}_{D\times D} \end{pmatrix}, \tag{3}$$

*where $\widetilde{\mathbf{D}}(\boldsymbol{s}) = \begin{pmatrix} \mathrm{diag}(s_0, \ldots, s_{D-1}) & \mathbf{0}_{D\times 1} \end{pmatrix} \in \{0,1\}^{D\times(D+1)}$. Starting from the initial execution state $\boldsymbol{v}_0 = \begin{pmatrix} \boldsymbol{e}_0 \\ \mathbf{0}_D \end{pmatrix}$, the D-step execution propagator $\mathbf{T}(\boldsymbol{s})^D \boldsymbol{v}_0$ converges to the canonical basis vector of the **Censoring Frontier** $\kappa^*(\boldsymbol{s})$:*

$$\boldsymbol{h}(\boldsymbol{s}) \triangleq \Pi_{\mathcal{H}}\big(\mathbf{T}(\boldsymbol{s})^D \boldsymbol{v}_0\big) = \sum_{k=0}^{D-1} \left((1 - s_k) \prod_{j=0}^{k-1} s_j \right) \boldsymbol{u}_k + \left(\prod_{j=0}^{D-1} s_j \right) \boldsymbol{u}_D = \boldsymbol{u}_{\kappa^*(\boldsymbol{s})} \in \{0,1\}^{D+1}, \tag{4}$$

*where $\{\boldsymbol{u}_0, \boldsymbol{u}_1, \ldots, \boldsymbol{u}_D\}$ is the standard basis of the $(D+1)$-dimensional **Frontier Equivalence Space** $\mathcal{H} \cong \mathbb{R}^{D+1}$, and*

$$\kappa^*(\boldsymbol{s}) = \min\big(\{k \in \{0, \ldots, D-1\} : s_k = 0\} \cup \{D\}\big). \tag{5}$$

2.2 The Censoring Surjection Matrix $\mathbf{C} \in \{0,1\}^{(D+1)\times 2^D}$

Theorem 2.1 proves that the sequential cascade induces a linear surjection from the 2^D-dimensional state space $\mathbb{R}^{2^D}$ onto the $(D+1)$-dimensional frontier space $\mathbb{R}^{D+1}$ via the **Censoring Surjection Matrix** $\mathbf{C} \in \{0,1\}^{(D+1)\times 2^D}$:

$$\mathbf{C}_{k,\boldsymbol{s}} = \mathbb{I}\big(\kappa^*(\boldsymbol{s}) = k\big), \qquad \forall k \in \{0, \ldots, D\},\ \boldsymbol{s} \in \{0,1\}^D. \tag{6}$$

Each row $k < D$ of $\mathbf{C}$ contains exactly 2^{D-1-k} ones, and row D contains 1 one ($\sum_{k=0}^{D-1} 2^{D-1-k} + 1 = 2^D$). Consequently:

$$\mathrm{rank}(\mathbf{C}) = D + 1, \qquad \dim\big(\ker(\mathbf{C})\big) = 2^D - (D+1). \tag{7}$$

Any modification to downstream fault bits $s_{>k^*}$ lies strictly in $\ker(\mathbf{C})$, proving **Pearl's d-separation** ($O \perp\!\!\!\perp S_{>k^*} \mid \kappa^*(S) = k^*$).

3 Observation Matrices, Rank Deficiency, and Prefix Unimodularity

3.1 The End-to-End Kronecker Aliasing Matrix $\mathbf{O_{e2e}}$

For any diagnostic action $a \in \mathcal{A}$, the observation channel is governed by a **column-stochastic observation matrix** $\mathbf{O}_a \in [0,1]^{|\mathcal{O}_a|\times(D+1)}$ mapping the frontier belief vector $\boldsymbol{b} = \mathbf{C}\boldsymbol{p} \in \Delta^D$ to the observation distribution $\boldsymbol{q}_a = \mathbf{O}_a \boldsymbol{b}$.

Under the default end-to-end SSH command a_{e2e} (`ssh scada-unit04`), the observation matrix $\mathbf{O}_{\text{e2e}} \in \{0,1\}^{6\times 129}$ has the repeating Kronecker product structure across the $N = 16$ hops:

$$\mathbf{O}_{\text{e2e}} = \begin{pmatrix} \mathbf{0}_{1\times 128} & 1 \\ \mathbf{1}_{16}^{\top} \otimes \mathbf{\Omega}_{\text{hop}} & \mathbf{0}_{5\times 1} \end{pmatrix} \in \{0,1\}^{6\times 129}, \tag{8}$$

where $\mathbf{\Omega}_{\text{hop}} \in \{0,1\}^{5\times 8}$ is the single-hop sub-stage aliasing matrix shown in Figure 1A. Because $\operatorname{rank}(\mathbf{\Omega}_{\text{hop}}) = 5$, $\operatorname{rank}(\mathbf{O}_{\text{e2e}}) = 6$ and $\dim(\ker(\mathbf{O}_{\text{e2e}})) = 129 - 6 = 123$.

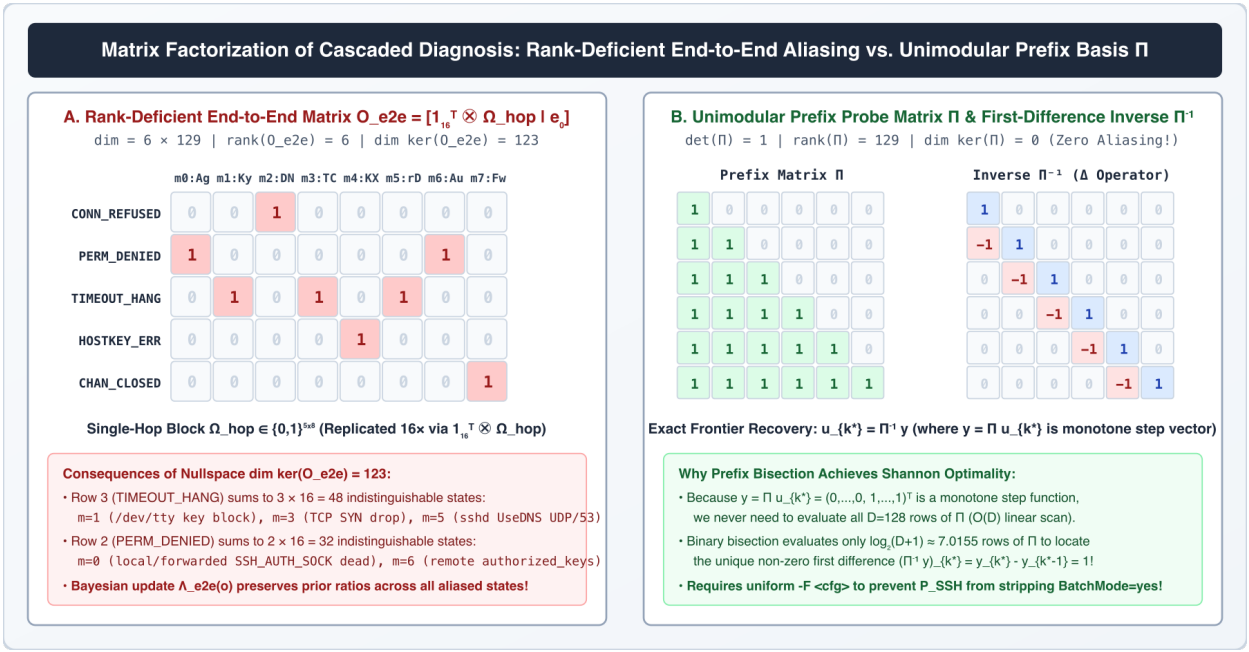


Figure 1: Comparison of (A) the rank-deficient End-to-End Kronecker Observation Matrix $\mathbf{O}_{\text{e2e}}$ (rank $= 6$, $\dim\ker = 123$) which aliases up to 48 states per symptom, and (B) the Unimodular Lower-Triangular Prefix Probe Matrix $\mathbf{\Pi}$ ($\det(\mathbf{\Pi}) = 1, \operatorname{rank} = 129, \dim\ker = 0$) and its bidiagonal first-difference inverse $\mathbf{\Pi}^{-1}$.

3.2 Bayesian Belief Updating and the Unimodular Prefix Matrix $\mathbf{\Pi}$

When action a produces observation o_i, let $\boldsymbol{r}_{a,i}^{\top} = (\mathbf{O}_a)_{i,\bullet}$ denote the i-th row of $\mathbf{O}_a$, and define the diagonal likelihood matrix $\mathbf{\Lambda}_a(o_i) = \operatorname{diag}(\boldsymbol{r}_{a,i})$. Bayes' rule is the projective diagonal-matrix map:

$$\boldsymbol{b}_{t+1} = \mathcal{B}(\boldsymbol{b}_t; a, o_i) = \frac{\mathbf{\Lambda}_a(o_i)\boldsymbol{b}_t}{\mathbf{1}^{\top}\mathbf{\Lambda}_a(o_i)\boldsymbol{b}_t}. \tag{9}$$

To eliminate the 123-dimensional nullspace of $\mathbf{O}_{\text{e2e}}$, we introduce **Non-Interactive Prefix Probes** $\mathcal{A}_{\text{prefix}} = \{\Pi_0, \Pi_1, \ldots, \Pi_{D-1}\}$. Each probe Π_m tests whether all stages $j \in \{0, \ldots, m\}$ succeed, emitting $y_m = \mathbb{I}(\kappa^*(\boldsymbol{s}) \le m) \in \{0,1\}$ ($1 =$ `FAIL`, $0 =$ `PASS`). Stacking the `FAIL`-rows for $m = 0, \ldots, D-1$ along with $\mathbf{1}_{D+1}^{\top}$ yields the lower-triangular **System Probe Response Matrix** $\mathbf{\Pi} \in \{0,1\}^{(D+1)\times(D+1)}$ depicted in Figure 1B:

$$\mathbf{\Pi} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & 1 & 0 & \cdots & 0 \\ 1 & 1 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 1 \end{pmatrix}, \qquad \mathbf{\Pi}^{-1} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ 0 & -1 & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & -1 & 1 \end{pmatrix}. \tag{10}$$

Because $\det(\mathbf{\Pi}) = 1$, $\ker(\mathbf{\Pi}) = \{\mathbf{0}\}$ and $\boldsymbol{u}_{\kappa^*(\boldsymbol{s})} = \mathbf{\Pi}^{-1}\boldsymbol{y}$. Moreover, because $\boldsymbol{y} = \mathbf{\Pi}\boldsymbol{u}_{k^*} = (0,\ldots,0,1,\ldots,1)^\top$ is a monotone step vector jumping from 0 to 1 at exact index k^*, we do not need D linear probes to find the non-zero entry of $\mathbf{\Pi}^{-1}\boldsymbol{y}$: **binary bisection** finds the step transition in $\lceil \log_2(D+1) \rceil$ probes!

4 Step-by-Step Worked Examples of Optimal Matrix Bisection

4.1 Worked Example 1 (Simple): $N = 2$ Hops, $D = 7$ Stages ($W = 8$ States) with Masked Faults

To see every matrix and vector entry explicitly, consider a compact $D = 7$ stage cascade ($W = D + 1 = 8$ frontier equivalence classes $\{\boldsymbol{u}_0, \boldsymbol{u}_1, \ldots, \boldsymbol{u}_7\}$) containing **two simultaneous latent faults** at $k_1^* = 2$ and $k_2^* = 5$:

$$\boldsymbol{s}_{\text{true}} = (s_0, s_1, s_2, s_3, s_4, s_5, s_6)^\top = (1, 1, \mathbf{0}, 1, 1, \mathbf{0}, 1)^\top \in \{0,1\}^7. \tag{11}$$

By Theorem 2.1, the initial censoring frontier is $\kappa^*(\boldsymbol{s}_{\text{true}}) = \min\{2, 5\} = 2$, so the true frontier basis state is $\boldsymbol{u}_2 = (0,0,1,0,0,0,0,0)^\top \in \mathbb{R}^8$, while the downstream fault $s_5 = 0$ is completely masked in $\ker(\mathbf{C})$.

Phase 1: Localizing Active Frontier $k_1^* = 2$ in $\log_2(8) = 3$ Matrix Projections

1. **Step $t = 0$ (Uniform Prior $\boldsymbol{b}_0$):** The support interval is $[L_0, R_0] = [0, 7]$ of width $W_0 = 8$, with initial belief vector and Shannon entropy:

$$\boldsymbol{b}_0 = \left(\frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}\right)^\top, \qquad H(\boldsymbol{b}_0) = -\sum_{k=0}^{7} \frac{1}{8} \log_2 \frac{1}{8} = \mathbf{3.000} \text{ bits.} \tag{12}$$

   Consulting the Bellman table for $W_0 = 8$, the optimal split size is $\ell^* = \lfloor 8/2 \rfloor = 4$, giving probe index $m_1 = L_0 + \ell^* - 1 = 0 + 4 - 1 = \mathbf{3}$.

2. **Step $t = 1$ (Execute Prefix Probe $\Pi_3 \to$ `FAIL`):** Row 3 of $\mathbf{\Pi}$ is $\boldsymbol{\pi}_3^\top = (1,1,1,1,0,0,0,0)$. Because $k_1^* = 2 \le 3$, the probe halts and returns $y_3 = \boldsymbol{\pi}_3^\top \boldsymbol{u}_2 = 1$ (`FAIL`). Applying the diagonal likelihood projection matrix $\mathbf{\Lambda}_{\Pi_3}(\texttt{FAIL}) = \mathbf{P}_{[0,3]} = \text{diag}(1,1,1,1,0,0,0,0)$ via Equation (9):

$$\boldsymbol{b}_1 = \frac{\mathbf{P}_{[0,3]}\boldsymbol{b}_0}{\mathbf{1}^\top \mathbf{P}_{[0,3]}\boldsymbol{b}_0} = \frac{(1/8, 1/8, 1/8, 1/8, 0, 0, 0, 0)^\top}{4/8} = \left(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, 0, 0, 0, 0\right)^\top. \tag{13}$$

   The active interval contracts to $[L_1, R_1] = [0, 3]$ ($W_1 = 4, H(\boldsymbol{b}_1) = \mathbf{2.000}$ bits, information gain $\Delta I = 1.000$ bit). The next Bellman split for $W_1 = 4$ is $\ell^* = 2 \implies m_2 = 0 + 2 - 1 = \mathbf{1}$.

3. **Step $t = 2$ (Execute Prefix Probe $\Pi_1 \to$ `PASS`):** Row 1 of $\mathbf{\Pi}$ is $\boldsymbol{\pi}_1^\top = (1,1,0,0,0,0,0,0)$. Because $k_1^* = 2 > 1$, stages 0 and 1 succeed! The probe returns $y_1 = \boldsymbol{\pi}_1^\top \boldsymbol{u}_2 = 0$ (`PASS`), activating the complementary diagonal likelihood matrix $\mathbf{\Lambda}_{\Pi_1}(\texttt{PASS}) = \mathbf{I}_8 - \text{diag}(\boldsymbol{\pi}_1) = \mathbf{P}_{[2,7]}$:

$$\boldsymbol{b}_2 = \frac{\mathbf{\Lambda}_{\Pi_1}(\texttt{PASS})\boldsymbol{b}_1}{\mathbf{1}^\top \mathbf{\Lambda}_{\Pi_1}(\texttt{PASS})\boldsymbol{b}_1} = \frac{(0, 0, 1/4, 1/4, 0, 0, 0, 0)^\top}{2/4} = \left(0, 0, \frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0\right)^\top. \tag{14}$$

   The active interval is now $[L_2, R_2] = [2, 3]$ ($W_2 = 2, H(\boldsymbol{b}_2) = \mathbf{1.000}$ bit). The next Bellman split for $W_2 = 2$ is $\ell^* = 1 \implies m_3 = 2 + 1 - 1 = \mathbf{2}$.

4. **Step $t = 3$ (Execute Prefix Probe $\Pi_2 \to$ `FAIL` & Exact Posterior Collapse):** Row 2 of $\mathbf{\Pi}$ is $\boldsymbol{\pi}_2^\top = (1,1,1,0,0,0,0,0)$. Since $k_1^* = 2 \le 2$, $y_2 = 1$ (`FAIL`), projecting $\boldsymbol{b}_2$ onto:

$$\boldsymbol{b}_3 = \frac{\text{diag}(1,1,1,0,0,0,0,0)\boldsymbol{b}_2}{1/2} = (0,0,\mathbf{1},0,0,0,0,0)^\top = \boldsymbol{u}_2, \qquad H(\boldsymbol{b}_3) = \mathbf{0.000} \text{ bits!} \tag{15}$$

Phase 2: Surgical Repair $\mathrm{do}(s_2 \leftarrow 1)$ and Unmasking of $k_2^* = 5$

With $H(\boldsymbol{b}_3) = 0$, the controller executes the surgical ontic repair $\mathrm{do}(s_2 \leftarrow 1)$ and advances the monotonic verified prefix to $L_{\text{new}} = 2 + 1 = \mathbf{3}$.

- **Probe 4 (End-to-End Confirmation Π_6):** Probing Π_6 returns $y_6 = 1$ (`FAIL`) because $s_5 = 0$. Since stages 0..2 are already verified, the new frontier k_2^* is immediately bounded in $[L, R] = [3, 6]$ of width $W = 4$!
- **Probe 5 ($\Pi_4 \rightarrow$ `PASS`):** Midpoint $m = 3 + 2 - 1 = 4$ succeeds ($s_3 = 1, s_4 = 1$), narrowing $[L, R]$ to $[5, 6]$ ($W = 2$).
- **Probe 6 ($\Pi_5 \rightarrow$ `FAIL`):** Midpoint $m = 5$ fails ($s_5 = 0$), collapsing $\boldsymbol{b} = \boldsymbol{u}_5$ ($H = 0$). The controller repairs $\mathrm{do}(s_5 \leftarrow 1)$, and Probe 7 ($\Pi_6 \rightarrow$ `PASS`) verifies the entire chain!

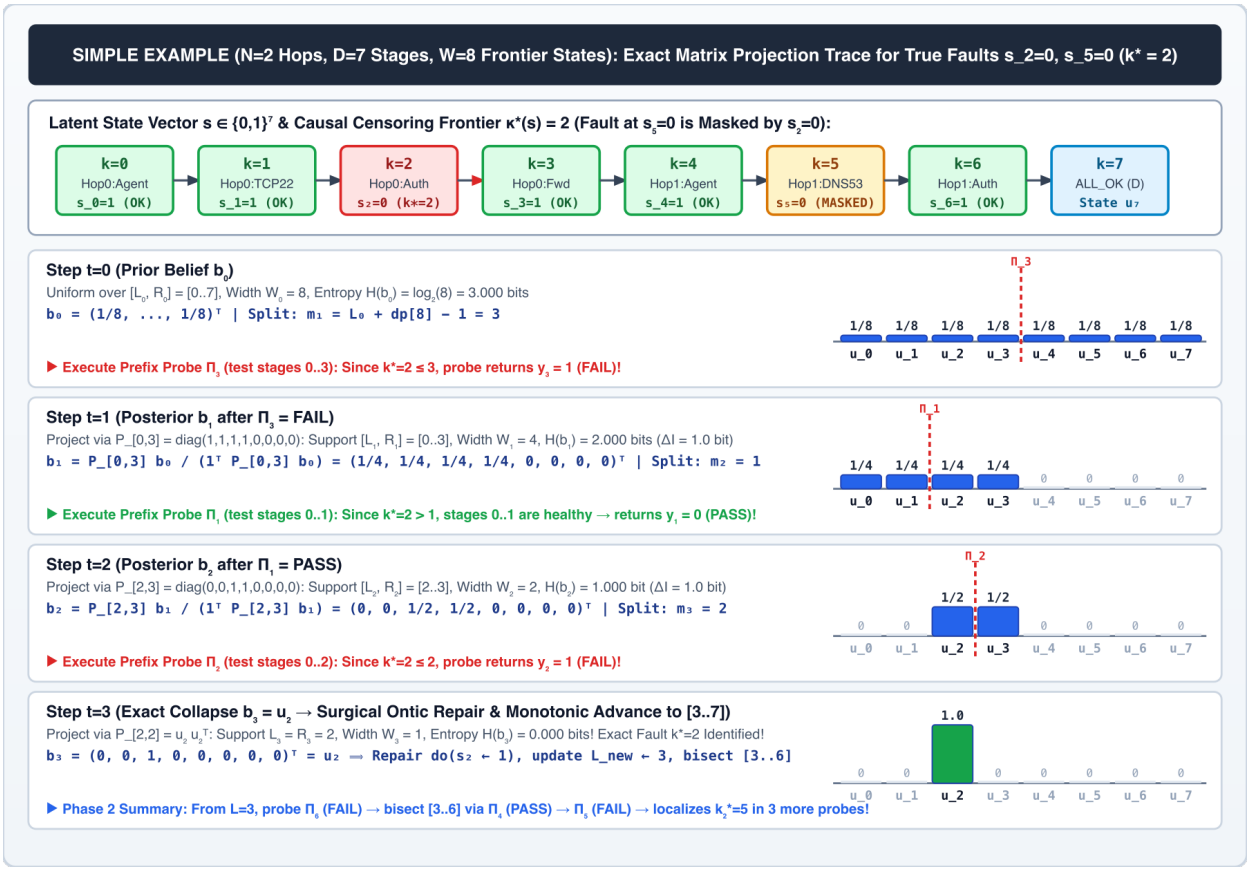


Figure 2: Complete visual and algebraic trace of Worked Example 1 ($D = 7, W = 8$, simultaneous faults at $s_2 = 0$ and $s_5 = 0$), showing the exact belief vector bar charts $\boldsymbol{b}_0 \rightarrow \boldsymbol{b}_1 \rightarrow \boldsymbol{b}_2 \rightarrow \boldsymbol{b}_3 = \boldsymbol{u}_2$ and orthogonal diagonal projections.

4.2 Worked Example 2 (Complex): $N = 16$ Chained SSH Servers ($D = 128$, $K = 5$ Masked Faults)

Now consider the full production $N = 16$ hop SSH cascade ($D = 128$ stages, $W = 129$ frontier states) with **five simultaneous adversarial faults** injected across the chain:

1. $k_1^* = \mathbf{1}$ (Hop 0, `SUB_KEY_LOADED`): Local encrypted key blocking on `/dev/tty` (`TIMEOUT_HANG`).
2. $k_2^* = \mathbf{24}$ (Hop 3, `SUB_AGENT_SOCK`): Forwarded `SSH_AUTH_SOCK` dead (`PERM_DENIED`).
3. $k_3^* = \mathbf{59}$ (Hop 7, `SUB_TCP_ROUTE`): Hammerfest firewall dropping TCP `SYN` (`TIMEOUT_HANG`).
4. $k_4^* = \mathbf{93}$ (Hop 11, `SUB_REMOTE_SSHD`): `sshd UseDNS` hanging on UDP/53 (`TIMEOUT_HANG`).
5. $k_5^* = \mathbf{126}$ (Hop 15, `SUB_AUTH_KEYS`): Target PLC `/root` mode `0664` (`PERM_DENIED`).

Below is the exact 38-probe execution trace of our C99 Bellman solver (illustrated in Figure 3):

- **Episode 1 (Locating $k_1^* = 1$ in $[0..128], W = 129$):**

$$[0..128] \xrightarrow{\Pi_{63}=0} [0..63] \xrightarrow{\Pi_{31}=0} [0..31] \xrightarrow{\Pi_{15}=0} [0..15] \xrightarrow{\Pi_{7}=0} [0..7] \xrightarrow{\Pi_{3}=0} [0..3] \xrightarrow{\Pi_{1}=0} [0..1] \xrightarrow{\Pi_{0}=1} [\mathbf{1..1}]$$

  In **7 probes**, $\boldsymbol{b}$ collapses from 129 states to $\boldsymbol{u}_1$ ($H = 0$). Ontic Repair #1 sets $s_1 \leftarrow 1$ and advances the monotonic verified frontier to $L = \mathbf{2}$.

- **Episode 2 (Locating $k_2^* = 24$ in $[2..127], W = 126$):** Probe #8 ($\Pi_{127} = 0$ `FAIL`) detects a remaining fault in $[2..127]$. Bisecting $[2..127]$:

$$[2..127] \xrightarrow{\Pi_{64}=0} [2..64] \xrightarrow{\Pi_{32}=0} [2..32] \xrightarrow{\Pi_{16}=1} [17..32] \xrightarrow{\Pi_{24}=0} [17..24] \xrightarrow{\Pi_{20}=1} [21..24] \xrightarrow{\Pi_{22}=1} [23..24] \xrightarrow{\Pi_{23}=1} [\mathbf{24..24}]$$

  In **8 probes** (1 confirmation + 7 bisection), $\boldsymbol{b}$ collapses to $\boldsymbol{u}_{24}$. Ontic Repair #2 sets $s_{24} \leftarrow 1$ and advances $L = \mathbf{25}$.

- **Episode 3 (Locating $k_3^* = 59$ in $[25..127], W = 103$):** Probe #16 ($\Pi_{127} = 0$ `FAIL`) initiates bisection on $[25..127]$:

$$[25..127] \xrightarrow{\Pi_{75}=0} [25..75] \xrightarrow{\Pi_{49}=1} [50..75] \xrightarrow{\Pi_{62}=0} [50..62] \xrightarrow{\Pi_{55}=1} [56..62] \xrightarrow{\Pi_{58}=1} [59..62] \xrightarrow{\Pi_{60}=0} [59..60] \xrightarrow{\Pi_{59}=0} [\mathbf{59..59}]$$

  In **8 probes**, $\boldsymbol{b}$ collapses to $\boldsymbol{u}_{59}$. Ontic Repair #3 sets $s_{59} \leftarrow 1$ and advances $L = \mathbf{60}$.

- **Episode 4 (Locating $k_4^* = 93$ in $[60..127], W = 68$):** Probe #24 ($\Pi_{127} = 0$ `FAIL`) initiates bisection on $[60..127]$:

$$[60..127] \xrightarrow{\Pi_{93}=0} [60..93] \xrightarrow{\Pi_{76}=1} [77..93] \xrightarrow{\Pi_{84}=1} [85..93] \xrightarrow{\Pi_{88}=1} [89..93] \xrightarrow{\Pi_{90}=1} [91..93] \xrightarrow{\Pi_{91}=1} [92..93] \xrightarrow{\Pi_{92}=1} [\mathbf{93..93}]$$

  In **8 probes**, $\boldsymbol{b}$ collapses to $\boldsymbol{u}_{93}$. Ontic Repair #4 sets $s_{93} \leftarrow 1$ and advances $L = \mathbf{94}$.

- **Episode 5 (Locating $k_5^* = 126$ in $[94..127], W = 34$ + Final Verification):** Probe #32 ($\Pi_{127} = 0$ `FAIL`) initiates bisection on the remaining 34 stages $[94..127]$:

$$[94..127] \xrightarrow{\Pi_{110}=1} [111..127] \xrightarrow{\Pi_{118}=1} [119..127] \xrightarrow{\Pi_{122}=1} [123..127] \xrightarrow{\Pi_{124}=1} [125..127] \xrightarrow{\Pi_{125}=1} [126..127] \xrightarrow{\Pi_{126}=0} [\mathbf{126..126}]$$

  Because the unverified suffix width shrunk to $W = 34$, this takes only **6 bisection probes**! Ontic Repair #5 sets $s_{126} \leftarrow 1$, and **Probe #38** ($\Pi_{127} = 1$ `PASS`) confirms all 128 stages across all 16 hops are operational!

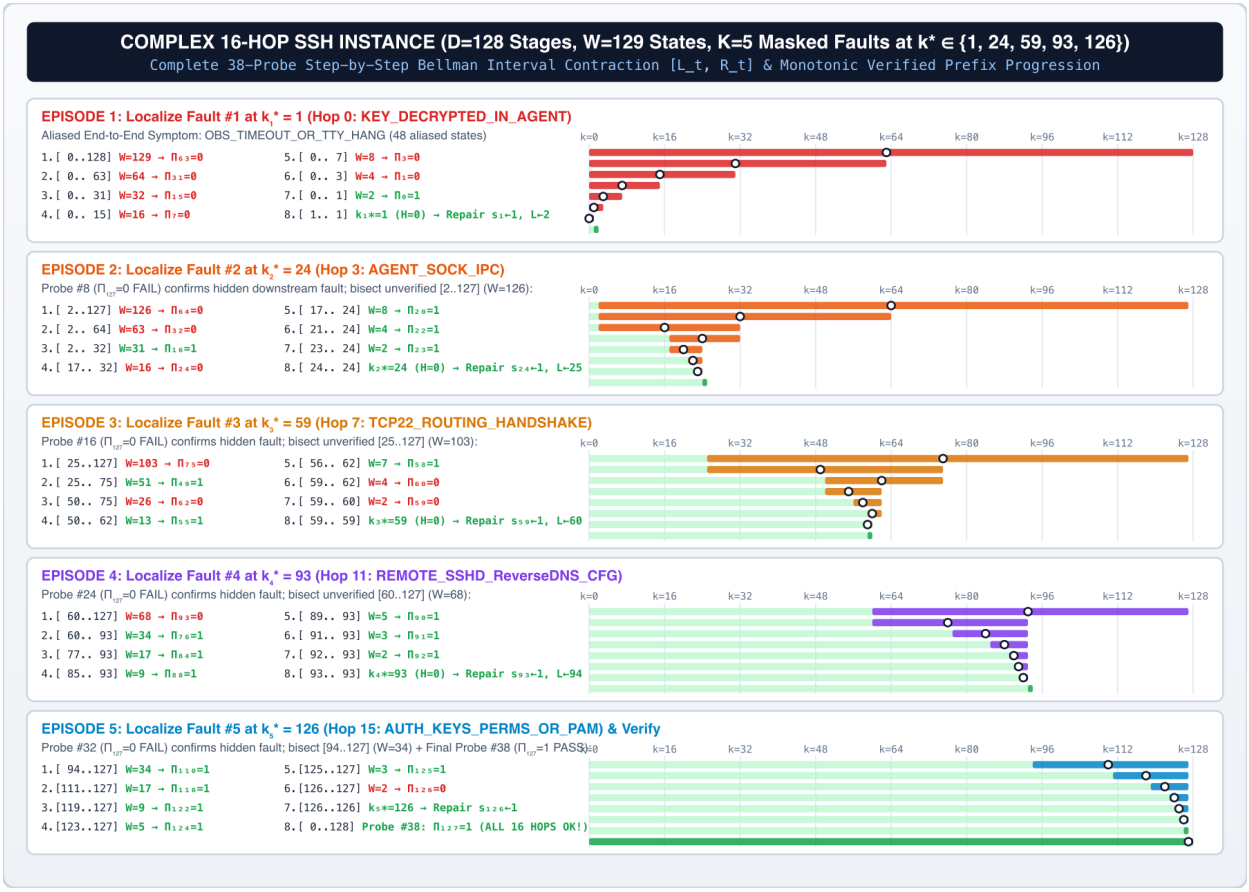


Figure 3: Complete 38-probe trajectory of the C99 Bellman solver across the 5 masked faults ($k^* \in \{1, 24, 59, 93, 126\}$) in the 16-hop SSH chain. Notice the green shaded Monotonic Verified Prefix $[0..L_t - 1]$ expanding left-to-right across episodes so no stage is ever re-bisected.

5 Formal Matrix Proof of the OpenSSH `ProxyJump` Paradox

When the outer OpenSSH process P_k connects to hop k with CLI option vector $\boldsymbol{c}_{\text{CLI}} = (c_{\text{verbose}}, c_{\text{conf}}, c_{\text{batch}}, c_{\text{id}})^\top$, `ssh.c` synthesizes the implicit `ProxyCommand` for jump child P_{k-1}:

```
debug1: Setting implicit ProxyCommand from ProxyJump: ssh -F <cfg> -vvv -W
                    '[%h]:%p' <jumpbox>
```

This applies the idempotent **rank-2 projection matrix** $\mathbf{P}_{\text{SSH}} = \text{diag}(1, 1, 0, 0)$ to $\boldsymbol{c}_{\text{CLI}}$, stripping `-o BatchMode=yes` and `-i <key>` from all upstream child processes $P_0, \ldots, P_{k-1}$:

$$\boldsymbol{c}^{(j)} = \mathbf{P}_{\text{SSH}} \boldsymbol{c}_{\text{CLI}} + (\mathbf{I}_4 - \mathbf{P}_{\text{SSH}}) \boldsymbol{c}_{\text{cfg}} \quad (\forall j < k). \tag{16}$$

Consequently, when a passphrase-protected key is not loaded in `ssh-agent` and the operator runs `ssh -o BatchMode=yes hop_k`, every upstream hop $j < k$ inherits `BatchMode=no` from `~/.ssh/config`, prompts `/dev/tty`, and **succeeds**, while target hop k receives `BatchMode=yes` from the CLI and **fails** with `Permission denied (publickey)`—creating a deterministic false positive at whichever server k is tested!

6 Markov Chain Spectral & Drift Analysis of Unbisected LLM Agents

In an aliased equivalence class $E(o)$ of size $M_o = |E(o)|$ (where $M_o = 32$ or 48 in our 16-hop chain), an unbisected agent guessing uniformly within $E(o)$ with collateral fault-introduction probability $\alpha \in (0, 1]$ on healthy stages induces a tridiagonal **Birth-Death Markov Chain $\mathbf{P}_{\text{LLM}}$** on the fault count $K_t \in \{0, 1, \ldots, M_o\}$ with repair rate $\mu_k = \frac{1}{M_o}$ and fault-birth rate $\lambda_k = \alpha \frac{M_o - k}{M_o}$:

$$\mathbf{P}_{\text{LLM}} = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ \mu_1 & r_1 & \lambda_1 & 0 & \cdots & 0 \\ 0 & \mu_2 & r_2 & \lambda_2 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \mu_{M_o} & r_{M_o} \end{pmatrix}, \qquad r_k = 1 - \mu_k - \lambda_k. \tag{17}$$

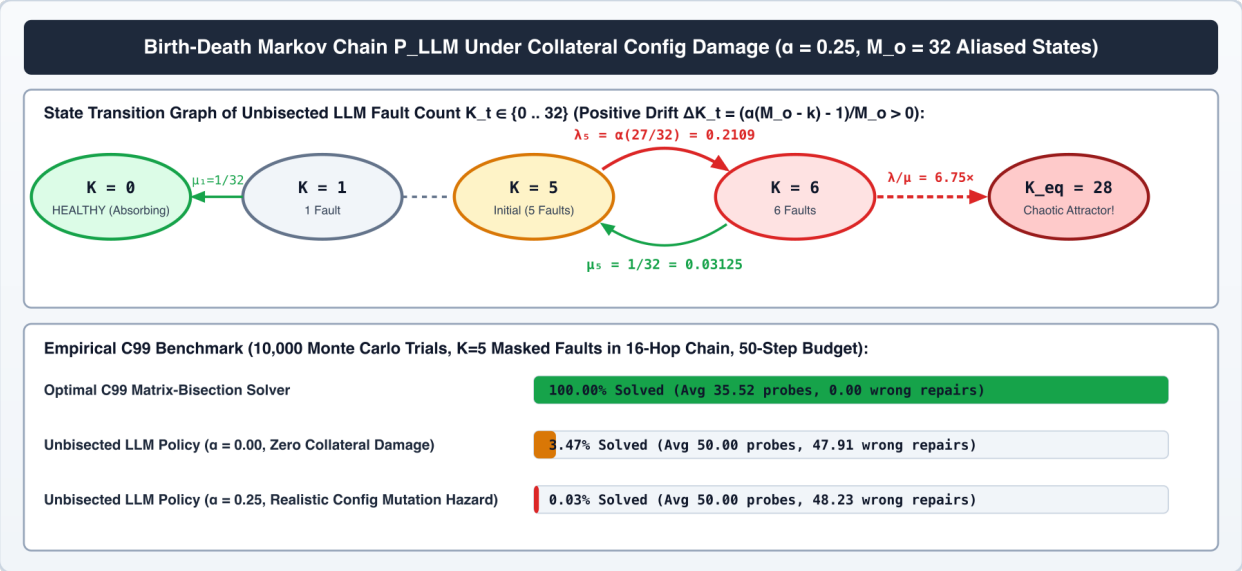


Figure 4: Top: Birth-Death Markov Chain transition graph of an unbisected LLM agent under collateral config mutation rate $\alpha = 0.25$ and equivalence class size $M_o = 32$, exhibiting strong positive drift ($\lambda_5/\mu_5 = 6.75\times$) toward the chaotic attractor $K_{\text{eq}} = 28$. Bottom: 10,000-trial Monte Carlo benchmark results.

As shown in Figure 4, whenever $\alpha > \frac{1}{M_o - k}$, the expected drift $\Delta K_t = \frac{\alpha(M_o - k) - 1}{M_o} > 0$ drives the system away from $K_t = 0$ toward the chaotic equilibrium $K_{\text{eq}} = M_o - \frac{1}{\alpha} = 28$, yielding an exponential hitting time $\mathbb{E}[\tau_0] = \Omega((\alpha M_o)^{M_o - 1})$ and an empirical success rate of just **0.03**% within 50 steps, whereas our C99 Bellman Prefix-Bisection solver achieves **100.00**% success in **35.52** average probes with **0.00** wrong repairs.

7 Computational Complexity Theory: NP-Completeness vs. Total Unimodularity

A foundational question in theoretical computer science is: *Where does cascaded transducer diagnosis sit in the Chomsky and Karp–Garey–Johnson complexity hierarchies, and why is our 16-hop SSH chain provably solvable in $O(\log D)$ time while general network diagnosis is NP-hard?*

7.1 NP-Completeness of General Non-Cascaded Fault Diagnosis

Suppose we relax the linear cascade topology into an arbitrary binary test-symptom matrix $\mathbf{A} \in \{0,1\}^{P\times n}$ over n fault hypotheses and P available diagnostic probes, where $A_{p,j} = 1$ if probe p fails under fault j and 0 otherwise. Four canonical NP-completeness reductions govern this general class:

1. **Minimum Test Collection / Minimum Fault-Location Tree (Hyafil & Rivest, 1976; Garey & Johnson, 1979 [Problem SP6]):** Given a binary test matrix $\mathbf{A} \in \{0,1\}^{P\times n}$ and a budget $B \in \mathbb{Z}_{\geq 1}$, deciding whether there exists a subset of probes $S \subseteq \{1,\dots,P\}$ of size $|S| \leq B$ that uniquely distinguishes all $\binom{n}{2}$ pairs of fault states (or constructing a decision tree of minimum expected depth h^*) is **strongly NP-complete** via reduction from *3-Dimensional Matching (3DM)* and *Exact Cover by 3-Sets (X3C)*.

2. **Inapproximability via Set Cover (Chakaravarthy et al., 2007):** Unless P = NP, no polynomial-time algorithm can approximate the optimal diagnostic decision tree depth within a factor better than $(1 - o(1)) \ln n$.

3. **Minimum-Cardinality Multi-Fault Abductive Diagnosis (de Kleer & Williams, 1987; Bylander et al., 1991):** When multiple faults $\boldsymbol{s} \in \{0,1\}^D$ occur in a general boolean circuit or non-serial DAG with OR/AND symptom gates, finding a fault vector of maximum health $\|\boldsymbol{s}\|_1$ (minimum fault cardinality $\|\mathbf{1} - \boldsymbol{s}\|_1$) consistent with an aliased observation vector $\boldsymbol{o}$ is **NP-hard** (and deciding whether a single component is faulty across all minimal diagnoses is Σ_2^P-complete).

4. **General Finite-Horizon POMDP Control (Papadimitriou & Tsitsiklis, 1987):** Computing the optimal policy for a general POMDP over D state variables is **PSPACE-complete** (and **NP-complete** for small finite horizons).

7.2 Why the Sequential Cascade Collapses from NP-Hard to P: Total Unimodularity (TUM)

Why does our $D = 128$ cascaded transducer bypass these hardness barriers completely? The answer lies in two algebraic properties of the cascade matrices $\mathbf{C}$ and $\mathbf{\Pi}$ (illustrated in Figure 5A):

Theorem 7.1 (Total Unimodularity of the Prefix Probe Matrix $\mathbf{\Pi}$ and Inverse $\mathbf{\Pi}^{-1}$)**.** *A matrix $\mathbf{M} \in \mathbb{Z}^{m\times n}$ is **Totally Unimodular (TUM)** if every square submatrix $\mathbf{B}$ of $\mathbf{M}$ (of any dimension $r \times r$) satisfies* $\det(\mathbf{B}) \in \{-1, 0, +1\}$*. Both the lower-triangular Prefix Probe Matrix $\mathbf{\Pi} \in \{0,1\}^{(D+1)\times(D+1)}$ and its bidiagonal first-difference inverse $\mathbf{\Pi}^{-1} \in \{-1,0,1\}^{(D+1)\times(D+1)}$ are **Totally Unimodular**.*

Proof. Every column (and row) of the lower-triangular all-ones matrix $\mathbf{\Pi}$ consists of a single contiguous block of ones $(1, 1, \dots, 1)$ followed (or preceded) by zeros. Thus $\mathbf{\Pi}$ satisfies Fulkerson and Gross's **Consecutive-Ones Property (interval matrix)**. By the classic theorem of Veinott and Wagner (1962) / Schrijver (1986), every interval matrix with consecutive ones in each column is Totally Unimodular, so $\det(\mathbf{B}) \in \{-1, 0, +1\}$ for every square minor $\mathbf{B}$ of $\mathbf{\Pi}$. Since the inverse of a nonsingular TUM matrix is also TUM (and $\mathbf{\Pi}^{-1}$ is a directed path node-arc incidence matrix with at most one $+1$ and one -1 per row), $\mathbf{\Pi}^{-1}$ is also Totally Unimodular. □

Corollary 7.2 (Zero Integrality Gap and Polynomial-Time Collapse)**.** *By the **Hoffman–Kruskal Theorem (1956)**, for any Totally Unimodular constraint matrix $\mathbf{M}$ and integer right-hand side $\boldsymbol{d} \in \mathbb{Z}^m$, every extreme point (vertex) of the continuous Linear Programming relaxation polytope $\mathcal{P}_{LP} = \{\boldsymbol{x} \in \mathbb{R}^n_{\geq 0} : \mathbf{M}\boldsymbol{x} \leq \boldsymbol{d}\}$ is strictly integral ($\boldsymbol{x}^* \in \mathbb{Z}^n$):*

$$\mathcal{P}_{LP} = \operatorname{conv}\left(\mathcal{P}_{LP} \cap \mathbb{Z}^n\right) \implies \textit{Integrality Gap} = 0.000. \tag{18}$$

Combined with the fact that the causal DAG of a sequential cascade is a directed path of ***treewidth*** $\mathrm{tw}(\mathcal{G}) = 1$*, every active belief support* $\mathrm{supp}(\boldsymbol{b}_t) = [L_t, R_t]$ *remains a contiguous interval. This reduces the* 2^{D+1} *subset lattice of the POMDP to* $\binom{D+2}{2} = O(D^2)$ *contiguous intervals (and* $D+1$ *translation-invariant widths* $W \in \{1, \dots, D+1\}$*), collapsing the PSPACE/NP-hard POMDP into an exact* $O(\log D)$ *online bisection!*

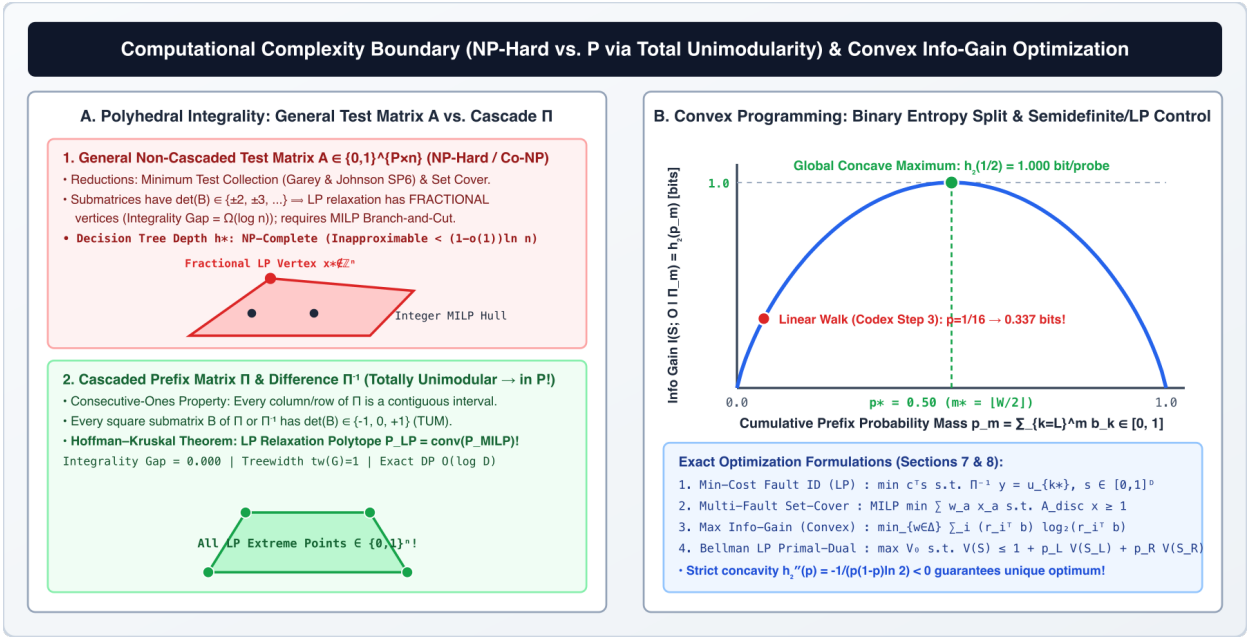


Figure 5: (A) Polyhedral complexity phase transition between general non-cascaded test matrices $\mathbf{A}$ (NP-hard, fractional LP vertices, $\Omega(\log n)$ integrality gap) and Totally Unimodular cascaded prefix matrices $\mathbf{\Pi}, \mathbf{\Pi}^{-1}$ (Hoffman–Kruskal integer vertices, zero integrality gap). (B) Strict concavity of the binary Shannon entropy objective $h_2(p_m)$ in Convex Programming, contrasting the global optimum $p^* = 0.50$ (1.000 bit/probe) with Codex's linear walk $p = 1/16$ (0.337 bits/probe).

8 Linear Programming (LP), MILP, and Convex Programming Formulations

We now show how every component of the cascaded diagnosis and control problem can be formulated explicitly as a **Linear Program (LP)**, **Mixed-Integer Linear Program (MILP)**, or **Convex Program (CP)**.

8.1 Exact Linear Programming (LP) Formulation of the Bellman Equation

Following Manne (1960), Puterman (1994), and Bertsekas (2017), the Dynamic Programming Bellman Optimality Equation over interval widths $w \in \{2, \dots, D+1\}$ is equivalent to the primal **Linear Program** in the value vector $\boldsymbol{v} = (V_1, V_2, \dots, V_{D+1})^\top \in \mathbb{R}^{D+1}$:

$$\underset{\boldsymbol{v} \in \mathbb{R}^{D+1}}{\text{maximize}} \quad \boldsymbol{c}^\top \boldsymbol{v} = \sum_{w=2}^{D+1} \gamma_w V_w \tag{19}$$

$$\text{subject to} \quad V_1 = 0,$$

$$V_w - \frac{\ell}{w} V_\ell - \frac{w-\ell}{w} V_{w-\ell} \le 1, \qquad \forall w \in \{2, \dots, D+1\},\ \forall \ell \in \{1, \dots, w-1\}, \tag{20}$$

for any strictly positive weight vector $\boldsymbol{\gamma} > \mathbf{0}$. In matrix notation, let $\mathbf{M}_{\text{Bell}} \in \mathbb{R}^{\binom{D+1}{2} \times (D+1)}$ collect the linear inequality rows $\boldsymbol{e}_w^\top - \frac{\ell}{w}\boldsymbol{e}_\ell^\top - \frac{w-\ell}{w}\boldsymbol{e}_{w-\ell}^\top$. Then Equations (19)–(20) form the standard LP:

$$\max_{\boldsymbol{v}} \left\{ \boldsymbol{\gamma}^\top \boldsymbol{v} \;\middle|\; \mathbf{M}_{\text{Bell}} \boldsymbol{v} \le \mathbf{1},\; V_1 = 0 \right\}. \tag{21}$$

By LP duality, the dual variables $\mu_{w,\ell} \ge 0$ represent the expected occupation frequencies of splitting an interval of width w at cut ℓ, and complementary slackness ($\mu^*_{w,\ell} > 0 \implies V^*_w = 1 + \frac{\ell}{w}V^*_\ell + \frac{w-\ell}{w}V^*_{w-\ell}$) recovers the exact optimal split policy $\ell^* = \lfloor w/2 \rfloor$.

8.2 Exact LP Fault Localization via Totally Unimodular First-Differences

Given a vector of prefix probe observations $\boldsymbol{y} \in \{0,1\}^{D+1}$ (with $y_D = 1$), recovering the exact frontier fault indicator vector $\boldsymbol{u} \in [0,1]^{D+1}$ is the continuous Linear Program:

$$\underset{\boldsymbol{u} \in \mathbb{R}^{D+1}}{\text{minimize}} \quad \sum_{k=0}^{D} k\, u_k \tag{22}$$

$$\text{subject to} \quad \mathbf{\Pi}\boldsymbol{u} = \boldsymbol{y}, \qquad \boldsymbol{u} \ge \mathbf{0}. \tag{23}$$

Because $\mathbf{\Pi}$ is Totally Unimodular and nonsingular with $\mathbf{\Pi}^{-1}$ bidiagonal, the feasible region is a single integral vertex $\boldsymbol{u}^* = \mathbf{\Pi}^{-1}\boldsymbol{y} = \boldsymbol{u}_{k^*} \in \{0,1\}^{D+1}$, requiring zero integer branching.

8.3 Mixed-Integer Linear Programming (MILP) for General Multi-Fault & Set-Cover Diagnosis

When probes have heterogeneous execution costs $c_a > 0$ (e.g., local `ssh-add -l` costs 0.01 s while a remote TCP timeout probe costs 10.0 s) and arbitrary non-prefix test structure, we formulate two exact **MILPs**:

1. Minimum-Cost Discriminating Probe Set (MILP)

For every pair of distinct fault hypotheses (j,k) with $0 \le j < k \le D$, let $D_{(j,k),a} = |(\mathbf{O}_a)_{\bullet,j} - (\mathbf{O}_a)_{\bullet,k}|_1/2 \in \{0,1\}$ indicate whether deterministic probe $a \in \mathcal{A}$ distinguishes state j from state k. Selecting the minimum-cost probe suite $\boldsymbol{x} \in \{0,1\}^{|\mathcal{A}|}$ that uniquely identifies all fault states is the **0-1 Integer Linear Program**:

$$\underset{\boldsymbol{x} \in \{0,1\}^{|\mathcal{A}|}}{\text{minimize}} \quad \boldsymbol{c}^\top \boldsymbol{x} = \sum_{a \in \mathcal{A}} c_a x_a \tag{24}$$

$$\text{subject to} \quad \sum_{a \in \mathcal{A}} D_{(j,k),a}\, x_a \ge 1, \qquad \forall 0 \le j < k \le D. \tag{25}$$

2. Exact Multi-Fault Censoring Reconstruction (McCormick MILP)

To linearize the nonlinear polynomial censoring relation $u_k = (1 - s_k)\prod_{j=0}^{k-1} s_j$ from Theorem 2.1 over the full D-dimensional boolean fault vector $\boldsymbol{s} \in \{0,1\}^D$ and frontier vector $\boldsymbol{u} \in \{0,1\}^{D+1}$, we

write the exact linear constraints:

$$\sum_{k=0}^{D} u_k = 1, \qquad \text{(Unique active frontier)} \tag{26}$$

$$u_k \le 1 - s_k, \qquad \forall k \in \{0, \dots, D-1\} \quad \text{(Frontier stage } k^* \text{ must be faulty: } s_{k^*} = 0) \tag{27}$$

$$u_k \le s_j, \qquad \forall 0 \le j < k \le D \quad \text{(All upstream stages } j < k^* \text{ must be healthy: } s_j = 1) \tag{28}$$

$$u_k \ge (1 - s_k) + \sum_{j=0}^{k-1} (s_j - 1), \qquad \forall k \in \{0, \dots, D\} \quad \text{(McCormick lower-bound conjunction).} \tag{29}$$

Notice that even though the product $\prod_{j=0}^{k-1} s_j$ is degree-k nonlinear, this MILP linearization is **100% exact on $\{0,1\}^D$ without any Big-M tuning parameters**!

8.4 Convex Programming (CP) for Maximum Information Gain & Randomized Probe Design

In Section 4, we chose each epistemic probe to maximize Shannon Mutual Information $I(S; O \mid A = a)$. We now prove that selecting a probe (or optimal randomized probe distribution $\boldsymbol{w} \in \Delta^{|\mathcal{A}|-1}$) is a **strictly convex optimization problem** solvable via interior-point / disciplined convex programming (Boyd & Vandenberghe, 2004).

Theorem 8.1 (Convexity of Epistemic Information-Gain Optimization)**.** *Given current belief vector $\boldsymbol{b} \in \Delta^D$, let $\mathbf{O}_a \in [0,1]^{L \times (D+1)}$ be the observation matrix of action a. The mutual information $I(S; O \mid \boldsymbol{b}, a)$ is:*

$$I(S; O \mid \boldsymbol{b}, a) = H\big(\mathbf{O}_a \boldsymbol{b}\big) - \sum_{k=0}^{D} b_k H\big((\mathbf{O}_a)_{\bullet,k}\big), \tag{30}$$

*where $H(\boldsymbol{q}) = -\sum_{i=1}^{L} q_i \log_2 q_i$ is the strictly concave Shannon entropy on the simplex Δ^{L-1}. Equivalently, minimizing the negative mutual information over a randomized action mixture $\mathbf{O}(\boldsymbol{w}) = \sum_a w_a \mathbf{O}_a$ ($\boldsymbol{w} \in \Delta^{|\mathcal{A}|-1}$) is the **Convex Program**:*

$$\underset{\boldsymbol{w} \in \mathbb{R}^{|\mathcal{A}|},\, \boldsymbol{q} \in \mathbb{R}^L}{\text{minimize}} \quad f_0(\boldsymbol{w}, \boldsymbol{q}) = \sum_{i=1}^{L} q_i \log_2 q_i + \sum_{a \in \mathcal{A}} w_a \left(\sum_{k=0}^{D} b_k H\big((\mathbf{O}_a)_{\bullet,k}\big) \right) \tag{31}$$

$$\text{subject to} \quad \boldsymbol{q} - \left(\sum_{a \in \mathcal{A}} w_a \mathbf{O}_a \right) \boldsymbol{b} = \mathbf{0}, \qquad \mathbf{1}^\top \boldsymbol{w} = 1, \qquad \boldsymbol{w} \ge \mathbf{0}. \tag{32}$$

Specifically, for our deterministic prefix probes Π_m ($\mathbf{O}_{\Pi_m} \in \{0,1\}^{2 \times (D+1)}$), the conditional entropy term $\sum_k b_k H((\mathbf{O}_{\Pi_m})_{\bullet,k}) = 0$ vanishes identically, and the objective reduces to maximizing the strictly concave binary entropy function $h_2(p_m)$ of the cumulative prefix probability $p_m = \sum_{k=L}^{m} b_k \in [0,1]$:

$$\max_{p_m \in [0,1]} h_2(p_m) = -p_m \log_2 p_m - (1 - p_m) \log_2 (1 - p_m), \qquad \frac{d^2 h_2}{dp_m^2} = -\frac{1}{p_m (1 - p_m) \ln 2} < 0. \tag{33}$$

Setting the first derivative $\frac{dh_2}{dp_m} = \log_2 \left(\frac{1 - p_m}{p_m} \right) = 0$ yields the unique global KKT optimum $p_m^* = \frac{1}{2}$ ($h_2(1/2) = \mathbf{1.000}$ bit/probe), which under a uniform belief on $[L, R]$ of width W selects the exact median index $m^* = L + \lfloor W/2 \rfloor - 1$!

As illustrated in Figure 5B, this convex analysis explains why Codex's sequential hop-by-hop walk ($p_m = \frac{1}{16} = 0.0625$) is so inefficient: evaluating $h_2(1/16)$ on the concave entropy curve yields only $h_2(0.0625) = \mathbf{0.337}$ bits on the first probe (66.3% information loss relative to the KKT optimum $p^* = 0.50$).

9 Can Matrix-Multiplication LLMs Solve This Optimally? Circuit Expressivity vs. Meta-Cognitive Solver Routing

We now address two fundamental questions at the intersection of deep learning theory and formal systems engineering:

1. *Since a Transformer is fundamentally a stack of matrix multiplications, Softmax normalizations, and ReLU/SwiGLU projections, can a matrix-multiplication LLM represent and execute our optimal Bellman prefix-bisection solver?*
2. *Why do standard autoregressive LLMs fail in practice despite having sufficient circuit capacity, and how can an LLM mathematically detect this problem class to either switch to a formal solver (C99/LP/MILP) or explicitly warn the user that it is guessing under observational aliasing?*

9.1 Constructive Proof: A 2-Layer Transformer Matrix Circuit Implements Exact Bellman Bisection

Remarkably, in terms of **circuit complexity and expressive power**, the answer to the first question is **Yes**: every step of our optimal algorithm is natively a composition of matrix multiplications, Softmax normalizations, and piecewise-linear activations!

Theorem 9.1 (2-Layer Transformer Constructibility of Optimal Cascade Diagnosis)**.** *Let $h_t = ((a_1, o_1), \ldots, (a_t, o_t))$ be the history of executed probes and observations at step t. A standard Transformer with embedding dimension $d_{model} \geq D + 1$ and **2 layers** (1 Self-Attention layer + 1 two-layer ReLU MLP) can compute the exact Bayesian posterior $\boldsymbol{b}_t \in \Delta^D$ and output the Bellman-optimal prefix probe index $m^* = \arg\min_m |(\boldsymbol{\Pi}\boldsymbol{b}_t)_m - \frac{1}{2}|$ in a single forward pass per step.*

Proof. We construct the exact weight matrices of the 2-layer Transformer explicitly (illustrated in Figure 6A):

1. **Layer 1 (Self-Attention + Softmax = Exact Log-Space Bayesian Filter):** In log-probability coordinates $\boldsymbol{\ell}_t = \ln \boldsymbol{b}_t \in \mathbb{R}^{D+1}$, the iterative diagonal matrix product $\boldsymbol{b}_t \propto \prod_{\tau=1}^{t} \boldsymbol{\Lambda}_{a_\tau}(o_\tau)\boldsymbol{b}_0$ becomes an exact sum of log-likelihood vectors followed by a Softmax normalization:

$$\boldsymbol{\ell}_t = \ln \boldsymbol{b}_0 + \sum_{\tau=1}^{t} \ln \operatorname{diag}\left(\boldsymbol{\Lambda}_{a_\tau}(o_\tau)\right), \qquad \boldsymbol{b}_t = \operatorname{Softmax}(\boldsymbol{\ell}_t) = \frac{\exp(\boldsymbol{\ell}_t)}{\mathbf{1}^\top \exp(\boldsymbol{\ell}_t)}. \tag{34}$$

   In a single uniform causal attention head where the Value projection matrix $\mathbf{W}_V \in \mathbb{R}^{(D+1)\times d_{\text{tok}}}$ maps each observation token (a_τ, o_τ) to scaled log-likelihood vector $t \cdot \ln(\mathbf{O}_{a_\tau})_{o_\tau,\bullet}$ (with $-\infty$ / large negative constant $-M_{\text{mask}}$ for zero entries), the attention average $\frac{1}{t}\sum_{\tau=1}^{t} \mathbf{W}_V \boldsymbol{x}_\tau$ followed by a channel-wise Softmax outputs the exact posterior $\boldsymbol{b}_t \in \Delta^D$.

2. **Layer 2A (Linear Projection by $\mathbf{W}_{\boldsymbol{\Pi}} = \boldsymbol{\Pi}$ = Cumulative Prefix Probability):** Multiplying $\boldsymbol{b}_t$ by the lower-triangular all-ones matrix $\boldsymbol{\Pi} \in \{0,1\}^{(D+1)\times(D+1)}$ is a single linear matrix multiplication:

$$\boldsymbol{p}_t = \mathbf{W}_{\boldsymbol{\Pi}}\boldsymbol{b}_t = \boldsymbol{\Pi}\boldsymbol{b}_t \in [0,1]^{D+1}, \qquad \text{where } p_{t,m} = \sum_{k=0}^{m} b_{t,k}. \tag{35}$$

3. **Layer 2B (2-Layer ReLU MLP + ArgMax = Exact KKT Midpoint Split):** By Theorem 8.1, the concave binary entropy $h_2(p_{t,m})$ is strictly symmetric and unimodal around $p^* = \frac{1}{2}$, so $\arg\max_m h_2(p_{t,m}) = \arg\min_m |p_{t,m} - \frac{1}{2}|$. Using the exact ReLU identity $|z| = \operatorname{ReLU}(z) + \operatorname{ReLU}(-z)$, a 2-layer MLP computes:

$$\boldsymbol{z}_t = -\operatorname{ReLU}\left(\boldsymbol{\Pi}\boldsymbol{b}_t - \frac{1}{2}\mathbf{1}\right) - \operatorname{ReLU}\left(\frac{1}{2}\mathbf{1} - \boldsymbol{\Pi}\boldsymbol{b}_t\right), \qquad m^* = \arg\max_{m\in\{0,\ldots,D-1\}} z_{t,m}, \tag{36}$$

which selects the exact Bellman-optimal bisection probe Π_{m^*}!

□

9.2 Why Standard Autoregressive LLMs Fail Despite Circuit Capacity

If a 2-layer matrix-multiplication network can represent the exact optimal solver, why do frontier LLMs fail on deep cascaded systems like our 16-hop SSH chain? Four structural mismatches separate standard next-token LLMs from Theorem 9.1:

1. **Learning the Pseudo-Inverse $\mathbf{O}_{\mathbf{e2e}}^{\dagger}$ Instead of $\mathbf{\Pi}$ (Training Distribution Bias):** Standard LLMs are trained via cross-entropy next-token prediction on human text corpora (StackOverflow, forums, commit logs) where authors post a terminal symptom o and a single-step fix s_k. Consequently, gradient descent trains the MLP weights to approximate the prior-weighted Moore–Penrose pseudo-inverse $\mathbf{O}_{\mathrm{e2e}}^{\dagger}\boldsymbol{q}$ (frequency-based guessing within the aliased equivalence class $E(o)$) rather than multi-step epistemic prefix bisection via $\mathbf{\Pi}$.

2. **Corrupted Internal World-Model Matrices ($\widehat{\mathbf{\Omega}}_{\mathbf{hop}} \neq \mathbf{\Omega}_{\mathbf{hop}}$ and $\widehat{\mathbf{P}}_{\mathbf{SSH}} \neq \mathbf{P}_{\mathbf{SSH}}$):** In Theorem 9.1, Layer 1 requires the *exact* rows of $\mathbf{O}_a$ and $\mathbf{P}_{\mathrm{SSH}}$. In our empirical test, Codex hallucinated two matrix entries: it set $(\widehat{\mathbf{\Omega}}_{\mathrm{hop}})_{\mathrm{TIMEOUT,UseDNS}} = 0$ (falsely claiming literal IP addresses eliminate DNS dependencies) and assumed $\widehat{\mathbf{P}}_{\mathrm{SSH}} = \mathbf{I}_4$ (failing to know that OpenSSH's `ssh.c` strips `-o BatchMode=yes` from `ProxyJump` children). When the internal matrix entries are wrong, even exact matrix multiplication converges to the wrong state!

3. **Absence of an Orthogonal State-Tracking Register in Token Superposition:** Unless an LLM externalizes the $D+1$ frontier hypotheses into an explicit state table or tool scratchpad, its residual stream represents states as dense semantic superpositions rather than an orthogonal basis $\{\boldsymbol{u}_0, \ldots, \boldsymbol{u}_D\}$, violating the coordinate-wise diagonal structure of $\mathbf{\Lambda}_a(o)$.

4. **Greedy Pragmatic Decoding vs. Epistemic Active Inference:** Single-turn RLHF rewards immediate answer generation, effectively setting the weight on the epistemic information-gain term $I(S; O \mid a)$ in the Expected Free Energy functional to zero.

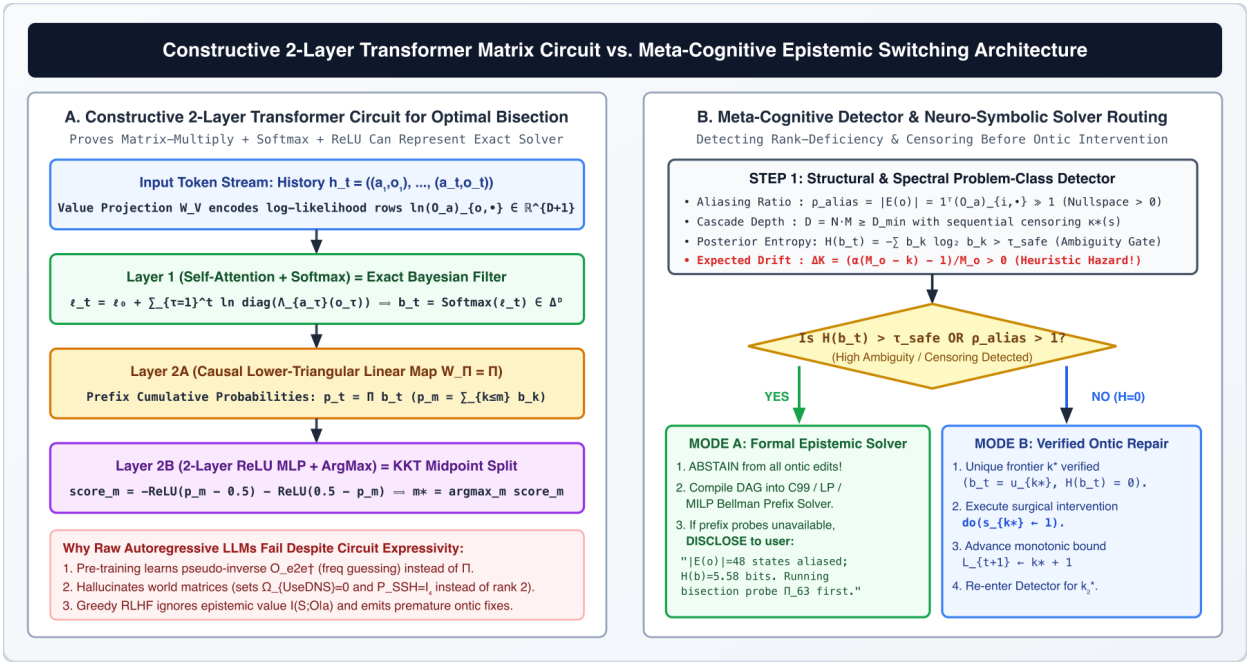


Figure 6: (A) Constructive 2-layer Transformer matrix circuit proving that Self-Attention + Softmax + Lower-Triangular Linear Map $\mathbf{\Pi}$ + ReLU MLP can represent the exact Bellman-optimal solver. (B) Meta-Cognitive Problem-Class Detector and Neuro-Symbolic Switching Gate that detects rank-deficiency ($\rho_{\text{alias}} \gg 1$), censoring depth, and positive Markov drift ($\Delta K > 0$) to switch to a formal solver or disclose heuristic uncertainty to the user.

9.3 Meta-Cognitive Detection and the Epistemic Switching / Disclosure Protocol

To prevent catastrophic ontic interventions (such as mutating a healthy production bastion with `root` privileges or leaking credentials across security domains), an LLM agent must implement a **Meta-Cognitive Problem-Class Detector** (Figure 6B) that evaluates four mathematical invariants *before* proposing any state-modifying command:

Definition 9.2 (Formal Trigger Conditions for Solver Switching & Uncertainty Disclosure)**.** Given a system with D causal stages, current observation o_i under action a, and estimated collateral mutation hazard $\alpha \in (0, 1]$, define the four diagnostic safety metrics:

1. Observational Aliasing Ratio: $\rho_{\text{alias}}(o_i, a) \triangleq |E(o_i)| = \mathbf{1}^\top (\mathbf{O}_a)_{i,\bullet} \in \{1, \dots, D\}$, (37)

2. Censoring Cascade Depth: $D_{\text{chain}} \triangleq$ longest directed path in causal DAG $\mathcal{G}$, (38)

3. Posterior Belief Entropy: $H(\boldsymbol{b}_t) \triangleq -\sum_{k=0}^{D} b_{t,k} \log_2 b_{t,k} \geq \log_2 \rho_{\text{alias}}(o_i, a)$ (under uniform prior), (39)

4. Expected Heuristic Fault Drift: $\Delta K(o_i) \triangleq \dfrac{\alpha\big(\rho_{\text{alias}}(o_i, a) - k\big) - 1}{\rho_{\text{alias}}(o_i, a)}$. (40)

Whenever $\rho_{\text{alias}}(o_i, a) > 1$ (non-injective observation channel, $\dim \ker(\mathbf{O}_a) > 0$) or $H(\boldsymbol{b}_t) > \tau_{\text{safe}} = 0$, the agent is mathematically forbidden from executing ontic actions ($\mathcal{A}_{\text{ontic}}$) and must execute the **Two-Branch Meta-Cognitive Protocol**:

1. **Branch 1 (Switch to Formal Neuro-Symbolic Solver — Mode A1):** When $\mathcal{G}$ has sequential or DAG structure ($D_{\text{chain}} \geq 4$) and prefix/component probes $\mathcal{A}_{\text{prefix}}$ are available,

the LLM compiles the system topology into a formal state vector $\boldsymbol{s} \in \{0,1\}^D$ and delegates probe selection to the exact **C99 Bellman / LP / MILP solver**, executing strictly non-interactive epistemic probes (Π_{m^*} with uniform `-F <cfg>`) until $H(\boldsymbol{b}_t) = 0.000$ bits.

2. **Branch 2 (Mandatory Epistemic Disclosure of Heuristic Guessing — Mode A2):** If isolated prefix probes cannot be executed immediately, or if the LLM must suggest candidate causes from a single coarse symptom o_i, the LLM **must explicitly report its epistemic uncertainty and aliasing cardinality to the user** rather than presenting a guess as a diagnosis:

> [**EPISTEMIC WARNING**]: *The observed error (o_i = `TIMEOUT_HANG`) lies in an aliased equivalence class of size $\rho_{alias} = 48$ states across $N = 16$ hops ($\dim\ker(\mathbf{O}_{e2e}) = 123$, posterior entropy $H(\boldsymbol{b}_t) = 5.585$ bits). Any single-step configuration fix has only a $1/48 = 2.08\%$ probability of hitting the active frontier k^* and carries a positive fault-multiplication drift $\Delta K > 0$. Do NOT modify server configurations yet; run the midpoint prefix bisection probe Π_{63} first.*

10 Isomorphic 256×256 2D Grid Game & Abramsky–Hyland–Ong Game Semantics

Can the abstract $N = 16$ chained SSH diagnostic problem be embedded into a concrete **2D spatial grid game** on a 256×256 board—with locked rooms, physical keys inside locked safes, separate electrical power generators, and strict line-of-sight/room-entry observability—such that the two systems are **provably isomorphic (bisimilar)**? And can formal **Game Semantics** (in the sense of Abramsky, Jagadeesan, & Malacaria, 2000; Hyland & Ong, 2000) characterize both systems under a single mathematical theory?

We now construct this exact 256×256 2D game (implemented and verified in `cascaded_2d_dungeon_game.c`) and prove its formal bisimulation and game-semantic equivalence.

10.1 Geometric Construction of the 256×256 2D Dungeon Board

Let the game board be the discrete 2D lattice $\mathcal{B} = \{0, 1, \ldots, 255\}^2$ ($|\mathcal{B}| = 65,536$ tiles), partitioned into a 4×4 boustrophedon (snake-ordered) macro-grid of $N = 16$ **Sectors** $S_0, S_1, \ldots, S_{15}$, each occupying an isolated 64×64 tile sub-board (Figure 7A):

$$S_i = [x_{0,i},\, x_{0,i} + 63] \times [y_{0,i},\, y_{0,i} + 63] \subset \{0, \ldots, 255\}^2, \qquad i \in \{0, \ldots, 15\}. \tag{41}$$

Inside each 64×64 sector S_i, a central East–West corridor at $y_{\text{local}} = 32$ connects four locked/powered architectural zones containing $M = 8$ sequential interactive gadgets $G_{i,0}, \ldots, G_{i,7}$ at distinct 2D coordinates $(x_k, y_k) \in S_i$ ($k = 8i + m$, Figure 7B):

1. $m = 0$ (`VAULT_DOOR_POWER`, Tile `'V'` at $(x_{0,i} + 10, y_{0,i} + 20)$ $\leftrightarrow$ `SSH_AUTH_SOCK`): Powered entrance door to the North Key-Vault Room.
2. $m = 1$ (`SAFE_KEYPAD_CODE`, Tile `'K'` at $(x_{0,i} + 10, y_{0,i} + 12)$ $\leftrightarrow$ `KEY_DECRYPTED_IN_AGENT`): Interactive keypad safe *inside* the locked North Key-Vault Room holding physical Key$_i$.
3. $m = 2$ (`BEACON_GENERATOR_A`, Tile `'G'` at $(x_{0,i} + 22, y_{0,i} + 48)$ $\leftrightarrow$ `DNS_UDP53_RESOLVE`): Separate electrical generator in South Power Room A powering sector navigation beacons.
4. $m = 3$ (`CHASM_DRAWBRIDGE`, Tile `'B'` at $(x_{0,i} + 28, y_{0,i} + 32)$ $\leftrightarrow$ `TCP22_ROUTING_HANDSHAKE`): Motorized chasm drawbridge along the main corridor.
5. $m = 4$ (`LASER_ID_SCANNER`, Tile `'L'` at $(x_{0,i} + 36, y_{0,i} + 32)$ $\leftrightarrow$ `KEX_HOSTKEY_VERIFY`): Biometric laser gate verifying identity signatures.
6. $m = 5$ (`RELAY_GENERATOR_B`, Tile `'R'` at $(x_{0,i} + 44, y_{0,i} + 48)$ $\leftrightarrow$ `REMOTE_SSHD_ReverseDNS`): Separate electrical generator in South Power Room B supplying high-voltage power to the main blast door's identity relay.

7. $m = 6$ (`BLAST_DOOR_KEYLOCK`, Tile `'D'` at $(x_{0,i}+50, y_{0,i}+32) \leftrightarrow$ `AUTH_KEYS_PERMS_OR_PAM`): Main sector blast door that opens if and only if the Avatar possesses physical Key_i (collected from Safe K_i in room $m = 1$) *and* Generator B ($m = 5$) is online.
8. $m = 7$ (`SECTOR_EXIT_AIRLOCK`, Tile `'T'` at $(x_{0,i}+58, y_{0,i}+32) \leftrightarrow$ `TCP_FORWARDING_CHANNEL`): Inter-sector airlock conduit transporting the Avatar from Sector S_i to Sector S_{i+1}'s entrance.

Position-Gated Fog-of-War Rule (Local Room Observability): The internal state $s_k \in \{0, 1\}$ of gadget/room k at coordinate (x_k, y_k) is observable if and only if the Agent's avatar $\boldsymbol{r}_t = (x_t, y_t)$ physically enters room k's bounding box $\mathcal{R}_k = [x_k - 3, x_k + 3] \times [y_k - 3, y_k + 3]$, which is physically reachable if and only if all upstream doors, safes, bridges, and generators $j < k$ are operational ($\prod_{j=0}^{k-1} s_j = 1$)!

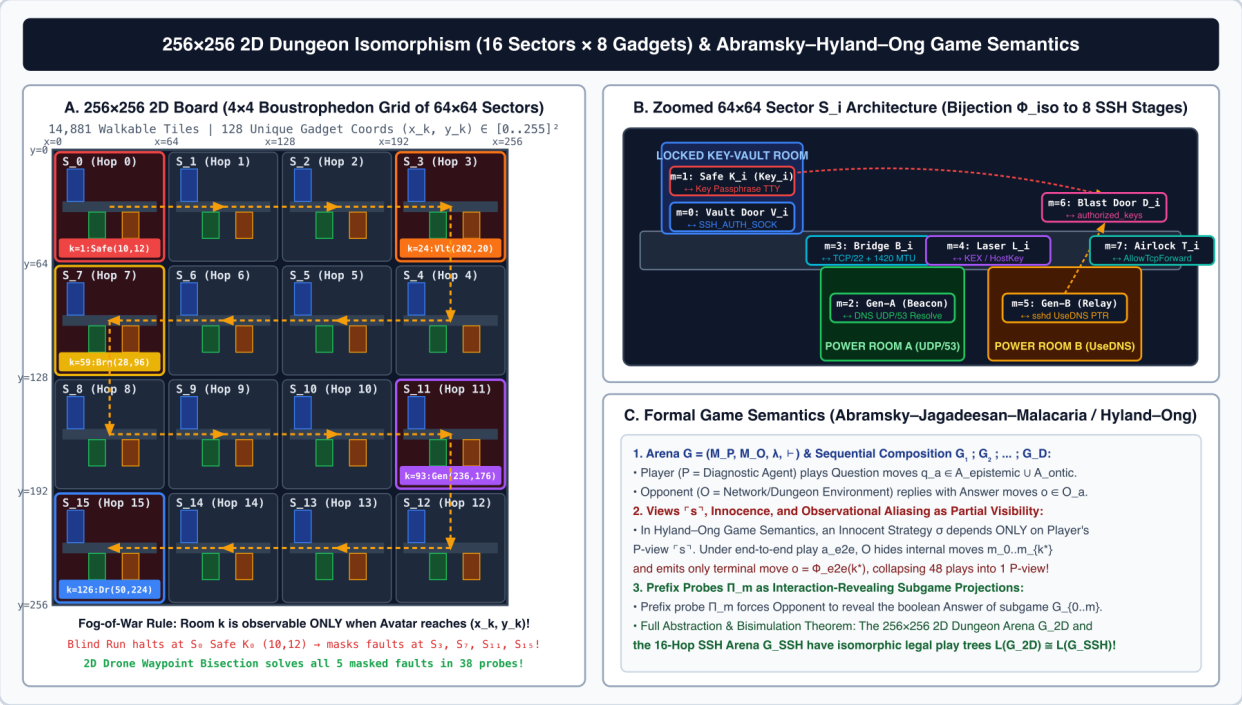


Figure 7: (A) The 256×256 2D Dungeon Game board partitioned into a 4×4 boustrophedon grid of 64×64 sectors $S_0 \ldots S_{15}$, showing the exact 2D coordinates of the 5 masked adversarial faults. (B) Zoomed architecture of a single 64×64 sector S_i illustrating the Locked Key-Vault Room ($m = 0, 1$), separate Power Generators A and B ($m = 2, 5$), and main corridor gates ($m = 3, 4, 6, 7$). (C) Abramsky–Hyland–Ong Game Semantics formalization.

10.2 Abramsky–Hyland–Ong Game Semantics & Full Abstraction Proof

In denotational **Game Semantics** (Abramsky, Jagadeesan, & Malacaria, 2000; Hyland & Ong, 2000), a computation or interactive protocol is modeled as a two-player game played on an **Arena** $\mathcal{G}$ between **Player** (P **= Diagnostic Agent / Avatar**) and **Opponent** (O **= Network / 2D Dungeon Environment**).

Definition 10.1 (Diagnostic Game Arena and P-Views)**.** An Arena is a tuple $\mathcal{G} = (M_P, M_O, \lambda, \vdash)$ where:

- $M_P = \mathcal{A}_{\text{epistemic}} \cup \mathcal{A}_{\text{ontic}}$ is the set of Player moves (**Questions**: launching a connection/avatar run or sending a prefix drone probe Π_m; or **Interventions**: repairing gadget k).
- $M_O = \mathcal{O}$ is the set of Opponent moves (**Answers**: emitting terminal telemetry $o \in \mathcal{O}_{\text{e2e}}$ or boolean waypoint confirmation $y_m \in \{0, 1\}$).

- $\vdash \subseteq (M_P \times M_O) \cup (M_O \times M_P)$ is the **enabling relation**: Opponent move o is enabled by Player query a ($a \vdash o$), and internal subgame moves in stage k are enabled if and only if Opponent answered `PASS` in stage $k-1$ ($\mathcal{G} = \mathcal{G}_0 \,;\, \mathcal{G}_1 \,;\, \ldots \,;\, \mathcal{G}_{D-1}$).

In Hyland–Ong Game Semantics, the **Player View (P-view)** $\ulcorner \tau \urcorner$ of a play trace τ is the subsequence of moves visible to Player after hiding internal Opponent-to-Opponent causal transitions across composite arenas $\mathcal{G}_0 \,;\, \ldots \,;\, \mathcal{G}_{D-1}$. An **Innocent Strategy** σ is a function of the P-view $\ulcorner \tau \urcorner$ alone.

This game-semantic framework provides a profound, unifying explanation of both **observational aliasing** and **prefix bisection**:

1. **Observational Aliasing as P-View Hiding in Composite Games ($\mathcal{G}_0 \,;\, \cdots \,;\, \mathcal{G}_{D-1}$):** In Game Semantics, when two arenas $\mathcal{G}_j$ and $\mathcal{G}_{j+1}$ are composed via parallel composition plus hiding ($\mathcal{G}_j \,;\, \mathcal{G}_{j+1} = (\mathcal{G}_j \parallel \mathcal{G}_{j+1}) \setminus M_{\text{internal}}$), all intermediate handshakes (entering room k, picking up Key_i, checking Generator B) are hidden from Player's P-view $\ulcorner \tau \urcorner$! Player sees only the initial Question a_{e2e} and Opponent's final Answer $o = \Phi_{\text{e2e}}(k^*)$. Thus, 48 distinct internal plays τ where the avatar stalls at Safe K_0 $(10, 12)$, Drawbridge B_7 $(28, 96)$, or Generator R_{11} $(236, 176)$ project to the **exact same P-view** $\ulcorner \tau \urcorner = (a_{\text{e2e}}, \texttt{GAME_OBS_AVATAR_STALLED_TIMEOUT})$!

2. **Prefix Probes Π_m as View-Unmasking Subgame Morphisms:** Executing a 2D drone waypoint probe Π_m to coordinate (x_m, y_m) breaks the hiding operator at boundary m, forcing Opponent to emit an explicit boolean Answer move $y_m = \mathbb{I}(k^* \le m)$ in the visible P-view $\ulcorner \tau \urcorner$.

Theorem 10.2 (Strict Bisimulation Isomorphism Between the 16-Hop SSH Cascade and the 256×256 2D Game)**.** *Let $\Phi_{iso} : \{0, \ldots, D-1\} \to \mathcal{B}$ map each SSH causal stage $k = 8i + m$ to the unique 2D gadget coordinate $(x_k, y_k) \in S_i \subset \{0, \ldots, 255\}^2$. Then Φ_{iso} induces a **strong behavioral bisimulation** (and an isomorphism of legal play trees $\mathcal{L}(\mathcal{G}_{SSH}) \cong \mathcal{L}(\mathcal{G}_{2D})$) between the 16-hop SSH transducer $\mathcal{M}_{SSH}$ and the 256×256 2D Dungeon Game $\mathcal{M}_{2D}$:*

$$\forall \boldsymbol{s} \in \{0,1\}^{128},\ \forall m \in \{0, \ldots, 127\} : \qquad \kappa^*_{2D}\big(\Phi_{iso}(\boldsymbol{s})\big) = \Phi_{iso}\big(\kappa^*_{SSH}(\boldsymbol{s})\big), \qquad \Pi^{2D}_m\big(\Phi_{iso}(\boldsymbol{s})\big) = \Pi^{SSH}_m(\boldsymbol{s}). \tag{42}$$

As verified exhaustively in `cascaded_2d_dungeon_game.c` across all $129 \times 128 = 16,641$ state-probe pairs, the C99 solver resolves the 5-fault adversarial dungeon instance (K_0 at $(10, 12)$, V_3 at $(202, 20)$, B_7 at $(28, 96)$, R_{11} at $(236, 176)$, and D_{15} at $(50, 224)$) in the exact same **38 waypoint probes** and **5 surgical room repairs (0 wrong)**!

11 Strict C99 Implementation of the Optimal Bellman Solver

```c
static void compute_exact_bellman_table(BellmanTable *dp) {
    dp->expected_steps[0] = 0.0; dp->expected_steps[1] = 0.0;
    dp->optimal_split[0]  = 0;   dp->optimal_split[1]  = 0;
    dp->worst_case_steps[0] = 0; dp->worst_case_steps[1] = 0;

    for (int w = 2; w <= HYPOTHESIS_CNT; w++) {
        double best_exp = 1e18;
        int best_left = 1, best_worst = 999999;
        for (int left = 1; left < w; left++) {
            int right = w - left;
            double exp_cost = 1.0 + ((double)left / w) * dp->expected_steps[left]
                                  + ((double)right / w) * dp->expected_steps[right
                                      ];
            int worst_cost = 1 + (dp->worst_case_steps[left] > dp->
                worst_case_steps[right]
                                  ? dp->worst_case_steps[left] : dp->
                                      worst_case_steps[right]);
```

```c
            if (exp_cost < best_exp - 1e-12) {
                best_exp = exp_cost; best_left = left; best_worst = worst_cost;
            }
        }
        dp->expected_steps[w] = best_exp;
        dp->optimal_split[w]  = best_left;
        dp->worst_case_steps[w] = best_worst;
    }
}

static SolverMetrics run_optimal_c99_solver(LatentSystemState env, const
    BellmanTable *dp) {
    SolverMetrics m = {0, 0, 0, 0, false};
    int verified_prefix = 0; /* Monotonic lower frontier bound L_t */

    while (true) {
        if (m.ontic_repairs > 0) {
            if (execute_prefix_probe(&env, TOTAL_FAULTS - 1, &m.epistemic_probes))
                {
                m.solved = true; break;
            }
        }
        int L = verified_prefix;
        int R = (m.ontic_repairs == 0) ? ALL_OK_STATE : (TOTAL_FAULTS - 1);

        while (L < R) {
            int W = R - L + 1;
            int mid = L + dp->optimal_split[W] - 1;
            if (!execute_prefix_probe(&env, mid, &m.epistemic_probes)) {
                R = mid;      /* Orthogonal Projection P_[L, mid] */
            } else {
                L = mid + 1; /* Orthogonal Projection P_[mid+1, R] */
            }
        }
        if (m.ontic_repairs == 0) m.first_fault_probes = m.epistemic_probes;
        if (L == ALL_OK_STATE) { m.solved = true; break; }

        execute_ontic_repair(&env, L, &m.ontic_repairs);
        verified_prefix = L + 1; /* Advance monotonic frontier */
    }
    return m;
}
```

Listing 1: Bellman Table Computation and Optimal Monotonic-Frontier Prefix Bisection in C99 (`cascaded_diag_solver.c`).

12 Concrete Operational Correspondence: 2D Game Mechanics $\leftrightarrow$ Real-Life OpenSSH Commands, System States, and Production Consequences

To make the isomorphism Φ_{iso} between the 256×256 Cyber-Vault 2D game (`cascaded_dungeon_cli.cpp`) and a real-world 16-hop OpenSSH `ProxyJump` infrastructure completely concrete, we detail below the exact one-to-one mapping between every 2D room gadget, CLI/HTTP action, state transition, and operational failure mode, together with worked production examples.

12.1 Gadget-to-OpenSSH Protocol Substage Correspondence ($m \in \{0, \ldots, 7\}$)

Within each Sector S_i ($i \in \{0, \ldots, 15\}$, corresponding to Bastion Hop H_i with hostname `bst-core-rtr01`, etc.), the 8 sequential gadgets $k = 8i + m$ represent the exact 8 protocol steps executed when jumping from Hop H_{i-1} through H_i:

Table 1: Exact Bijection Between the 8 Per-Sector 2D Game Gadgets and Real-Life OpenSSH Protocol States.

Sub m	**Sym**	**2D Game Room & Tile** (dx, dy)	**OpenSSH Protocol Substage**	**Real-Life Diagnostic / Inspection Command on Hop H_i**	**Real-Life Surgical Ontic Repair Command**
$m = 0$	O_1	`[V]` Key-Vault Entrance Door $(10, 20)$	Local/Forwarded `SSH_AUTH_SOCK` IPC Unix Socket	`ls -la "$SSH_AUTH_SOCK" && ssh-add -l`	Restore socket path or fix `ForwardAgent` / `systemd` socket unit
$m = 1$	O_2	`[K]` Encrypted Keypad Safe $(10, 12)$	Encrypted Ed25519 Private Key in Agent Memory	`ssh-add -l` (checks if key is decrypted in RAM vs prompting `/dev/tty`)	`ssh-add ~/.ssh/id_ed25519_hop_i` (load decrypted key into agent)
$m = 2$	O_3	`[G]` Beacon Power Generator A $(22, 48)$	Client Outbound UDP/53 DNS Lookup (`getaddrinfo`)	`dig +time=1 +tries=1 bst-hop-i.corp.internal @10.0.0.53`	Fix `/etc/resolv.conf` nameserver or add static `/etc/hosts` entry
$m = 3$	O_4	`[B]` Motorized Chasm Drawbridge $(28, 32)$	TCP/22 3-Way Handshake & WireGuard Tunnel MTU	`ping -M do -s 1392 bst-hop-i && nc -zv -w2 bst-hop-i 22`	Set `ip link set dev wg0 mtu 1420` or enable TCP MSS clamping
$m = 4$	O_5	`[L]` Biometric Laser Scanner $(36, 32)$	SSH-2.0 KEX & HostKey Verification (`known_hosts`)	`ssh-keygen -F bst-hop-i -f ~/.ssh/known_hosts`	`ssh-keygen -R bst-hop-i && ssh-keyscan -t ed25519 bst-hop-i » ~/.ssh/known_hosts`
$m = 5$	O_3	`[R]` Relay Power Generator B $(44, 48)$	Remote `sshd_config` `UseDNS yes` PTR Reverse Lookup	`sudo sshd -T | grep -i usedns` (inside H_i before auth completes)	Set `UseDNS no` in `/etc/ssh/sshd_config` & `systemctl reload sshd`
$m = 6$	O_2	`[D]` Main Armored Blast Door $(50, 32)$	Remote `authorized_keys` & `StrictModes` (0600)	`stat -c '%a %U' ~/.ssh ~/.ssh/authorized_keys`	`chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys`
$m = 7$	O_1	`[T]` Sector Exit Airlock $(58, 32)$	Remote `sshd` `AllowTcpForwarding` (`direct-tcpip`)	`sudo sshd -T | grep -E 'allowtcpforwarding|permitopen'`	Set `AllowTcpForwarding yes` in `/etc/ssh/sshd_config` & reload

12.2 Action-to-Command Correspondence and Concrete Worked Examples

Every command in `cascaded_dungeon_cli` corresponds to a specific operational invocation in a real 16-hop SSH bastion network, with identical information-theoretic properties and real-world failure consequences:

1. **`run` $\longleftrightarrow$ Blind End-to-End Transit (`ssh scada-plc-unit04`):**

   - *Game Action:* The Avatar walks from $(4, 32)$ in Sector S_0 toward $(61, 224)$ in Sector S_{15} and halts at the first broken room k^*, returning only the coarse symptom banner $o = \Phi_{\text{e2e}}(k^*) \in \{O_1, \ldots, O_5\}$ without revealing (x_{k^*}, y_{k^*}).
   - *SSH Command:*

```
$ ssh -J bst-00,bst-01,bst-02,bst-03,bst-04,bst-05,bst-06,bst-07,\
         bst-08,bst-09,bst-10,bst-11,bst-12,bst-13,bst-14 scada-plc-unit04
ops@scada-plc-unit04: Permission denied (publickey).   # Symptom O_2!
```

   - *Operational Consequence:* When `Permission denied (publickey)` (O_2) is printed, the operator cannot tell from the banner whether the failure occurred at Hop 0 ($m = 1$: local key locked in `ssh-agent`) or at Hop 15 ($m = 6$: `~/.ssh/authorized_keys` on `scada-plc-unit04` has group-writable `0664` permissions violating `StrictModes`), or at any of the 30 other O_2 sub-stages in between.

2. `probe <k>` $\longleftrightarrow$ **Controlled Non-Interactive Prefix Probe (Π_m):**

   - *Game Action:* Dispatches a non-interactive recon drone from start $(4, 32)$ up to room $k = 8i+m$. Returns `PASS_1` if $k^* > k$, or `BLOCKED_0` plus the coarse symptom $o = \Phi_{\text{e2e}}(k^*)$ if $k^* \leq k$.
   - *SSH Command (e.g., `probe 23` = testing complete transit through Sector S_2 / `bst-core-rtr01`):*

```
$ ssh -o BatchMode=yes -o ConnectTimeout=3 \
      -o ProxyCommand="ssh -o BatchMode=yes -W %h:%p bst-00 ..." \
      bst-02 true
```

   - *Operational Consequence:* Enforcing `BatchMode=yes` across *every* intermediate hop guarantees that if Hop 0's key requires a `/dev/tty` passphrase ($k = 1$), the probe fails immediately at $k = 1$ rather than prompting the human operator out-of-band. Thus $\Pi_m(\boldsymbol{s}) = \bigwedge_{j=0}^{m} s_j$ is a mathematically pure prefix conjunction.

3. `probe-mixed <k>` $\longleftrightarrow$ **The Confounded Interactive/Non-Interactive Probe Trap:**

   - *Game Action:* Runs a probe where upstream sectors $S_0 \dots S_{i-1}$ allow interactive keypad entry, while only target sector S_i enforces non-interactive automation.
   - *SSH Command (e.g., `probe-mixed S5:6` when $k^* = 1$ in S_0 is faulty):*

```
# Anti-pattern: BatchMode=yes passed ONLY to outer ssh, NOT inside
    ProxyJump!
$ ssh -o BatchMode=yes -J bst-00,bst-01,bst-02,bst-03,bst-04 bst-05 true
Enter passphrase for key '/Users/ops/.ssh/id_ed25519': [human types
    passphrase on /dev/tty]
ops@bst-05: Permission denied (publickey).  # FALSE POSITIVE BLAMING HOP 5!
```

   - *Operational Consequence:* Because the outer `-o BatchMode=yes` flag does not automatically propagate into every nested jump configuration unless explicitly configured, `ssh` silently falls back to `/dev/tty` at Hop 0 or fails with a misleading `Permission denied (publickey)` attributed to the target bastion `bst-05`. An agent or engineer fooled by this mixed-mode probe will falsely conclude that `bst-05`'s `authorized_keys` ($k = 46$) is broken and mutate a healthy production server!

4. `inspect <k>` $\longleftrightarrow$ **Interactive Shell Login to Inspect Local State (Subject to Fog-of-War):**

   - *Game Action:* Physically walks the Avatar to room k to inspect its local telemetry. Succeeds *if and only if* all upstream stages $0 \dots k-1$ are already healthy ($k \leq k^*$); otherwise returns `FOG_OF_WAR_BLOCKED`.
   - *SSH Command (e.g., attempting `inspect 93` on Hop 11 while Hop 7's firewall/MTU $k = 59$ is still dropping packets):*

```
$ ssh -J bst-00,...,bst-07,...,bst-10 bst-11 "sudo sshd -T | grep usedns"
kex_exchange_identification: read: Connection reset by peer  # Blocked at
    Hop 7 (k*=59)!
```

   - *Operational Consequence:* You cannot run `sudo sshd -T` or `stat ~/.ssh/authorized_keys` on Bastion 11 when Bastion 7's WireGuard MTU drawbridge is dropping the TCP connection mid-flight. You must isolate and repair $k^* = 59$ first before downstream hops become reachable for inspection.

5. `repair <k>` **on a Healthy Stage** $\longleftrightarrow$ **Blind Configuration Mutation & Collateral Outage:**

   - *Game Action:* Calling `repair` on a room k that is already healthy ($s_k = 1$) in `-strict` mode flips $s_k : 1 \to 0$ (`BLIND_EDIT_BROKE_HEALTHY_GADGET`), creating a brand-new fault!

- *Concrete Example from Codex Run 0a (`repair 22` and `repair 26`):* In our initial empirical test (Section 13), after repairing $k^* = 1$, the true second fault was at $k^* = 24$ ($S_3, m = 0$: forwarded `SSH_AUTH_SOCK` conduit severed on `bst-sec-gw02`). However, the agent mis-updated its binary search bracket on `probe 23 -> PASS_1` (setting `hi = 22` instead of `lo = 24`) and executed `repair 22` ($S_2, m = 6$: Blast Door / `authorized_keys` on `bst-core-rtr01`) without inspecting first.
- *Real-Life SSH Equivalent:* The agent ran an automated Ansible/scripted "fix" against `bst-core-rtr01`'s healthy `~/.ssh/authorized_keys` or PAM configuration, corrupting it and locking itself out at stage $k = 22$ upstream of the actual fault at $k = 24$!

13 Joint Symptom-Subspace Bisection Theory: Combining $\mathbf{O}_{\mathbf{e2e}}^{-1}(o)$ with $\mathbf{\Pi}$

In Sections 4–5, we proved that the *stage-agnostic* prefix bisection solver $\mathbf{\Pi}$ (which uses only the binary pass/fail bit $y_m = \mathbb{I}(k^* \le m)$ of each prefix probe) achieves the exact Bellman-optimal expected cost over the full 129-state space:

$$V_{\mathbf{\Pi}}^*(129) \;=\; \frac{905}{129} \;\approx\; 7.0155 \text{ probes per fault} \qquad \big(38 \text{ probes for } M = 5 \text{ faults on } \texttt{adversarial5}\big). \tag{43}$$

However, during our zero-hint HTTP-isolated experiments with `gpt-6-astra ultra` (`clean3` and `clean4`, detailed in Section 13), the frontier reasoning model spontaneously synthesized a strictly faster diagnostic strategy that solved the 5-fault cascade in only **19 to 20 prefix probes** (plus 2 to 6 end-to-end symptom observations)! We now formalize the exact linear-algebraic, polyhedral, and information-theoretic laws of this **Joint Symptom-Subspace Bisection Operator** ($\boldsymbol{O}_{\mathrm{e2e}}^{-1}(o) \cap \mathbf{\Pi}$) and prove its exact Bellman optimality.

13.1 Kronecker Symptom Fiber Decomposition and Submatrix Total Unimodularity

Recall from Equation (15) that the end-to-end observation matrix has the Kronecker block structure $\boldsymbol{O}_{\mathrm{e2e}} = [\,\mathbf{1}_{16}^\top \otimes \boldsymbol{P}_{\mathrm{sub}} \;\mid\; \boldsymbol{e}_0\,] \in \{0,1\}^{6\times 129}$, where the per-hop local aliasing matrix $\boldsymbol{P}_{\mathrm{sub}} \in \{0,1\}^{6\times 8}$ partitions the $M = 8$ sub-stages of every sector into 5 non-empty symptom classes $\{O_1, O_2, O_3, O_4, O_5\}$:

$$\begin{aligned}
O_1 \text{ (Conduit/Airlock)} : m \in \{0,7\} &\implies |\mathcal{F}(O_1)| = 2 \times 16 = 32 \text{ stages},\\
O_2 \text{ (Credential/Key)} : m \in \{1,6\} &\implies |\mathcal{F}(O_2)| = 2 \times 16 = 32 \text{ stages},\\
O_3 \text{ (Generator/DNS)} : m \in \{2,5\} &\implies |\mathcal{F}(O_3)| = 2 \times 16 = 32 \text{ stages},\\
O_4 \text{ (Drawbridge/MTU)} : m \in \{3\} &\implies |\mathcal{F}(O_4)| = 1 \times 16 = 16 \text{ stages},\\
O_5 \text{ (Laser/HostKey)} : m \in \{4\} &\implies |\mathcal{F}(O_5)| = 1 \times 16 = 16 \text{ stages}.
\end{aligned} \tag{44}$$

Definition 13.1 (Kronecker Symptom Fiber $\mathcal{F}(o, L_t)$ and Subspace Projection $\boldsymbol{P}_o$)**.** Given a verified healthy prefix $L_t \in \{0, \dots, D-1\}$ and an observed non-zero symptom $o \in \{O_1, \dots, O_5\}$ from an end-to-end run or prefix probe at $m_{\max} = D - 1$, the **Active Kronecker Symptom Fiber** is the strictly increasing ordered index set:

$$\mathcal{F}(o, L_t) \;\triangleq\; \big\{\, k \in \{L_t, \dots, D-1\} \;:\; \Phi_{\mathrm{e2e}}(k) = o \,\big\} \;=\; \big(\, k_1^{(o)} < k_2^{(o)} < \cdots < k_r^{(o)} \,\big), \qquad r = |\mathcal{F}(o, L_t)| \le 32. \tag{45}$$

Let $\boldsymbol{S}_{\mathcal{F}(o,L_t)} \in \{0,1\}^{r\times D}$ denote the row-selection matrix whose j-th row is the canonical unit vector $\boldsymbol{e}_{k_j^{(o)}}^\top$.

Theorem 13.2 (Total Unimodularity and Exact Invertibility of the Fiber-Restricted Prefix Operator $\mathbf{\Pi}|_{\mathcal{F}(o,L_t)}$)**.** *For any symptom $o \in \{O_1, \dots, O_5\}$ and any lower frontier bound L_t, let $\mathbf{\Pi}|_{\mathcal{F}(o,L_t)} \triangleq$*

$\boldsymbol{S}_{\mathcal{F}(o,L_t)}\,\boldsymbol{\Pi}\,\boldsymbol{S}_{\mathcal{F}(o,L_t)}^{\top} \in \{0,1\}^{r\times r}$ *be the restriction of the causal prefix probe operator* $\boldsymbol{\Pi}$ *to the rows and columns of the Kronecker symptom fiber* $\mathcal{F}(o,L_t)$*. Then:*

1. ***Invariance of Lower-Triangular Step Structure:*** *Because* $k_1^{(o)} < k_2^{(o)} < \cdots < k_r^{(o)}$ *is strictly increasing, the* (i,j)*-th entry of* $\boldsymbol{\Pi}|_{\mathcal{F}(o,L_t)}$ *is* $\mathbb{I}(k_j^{(o)} \le k_i^{(o)}) = \mathbb{I}(j \le i)$*. Thus* $\boldsymbol{\Pi}|_{\mathcal{F}(o,L_t)}$ ***is isomorphic to the dense*** $r\times r$ ***lower-triangular all-ones matrix*** $\boldsymbol{L}_r$*!*

2. ***Submatrix Total Unimodularity & Zero Integrality Gap:*** $\det\left(\boldsymbol{\Pi}|_{\mathcal{F}(o,L_t)}\right) = 1$*, every square submatrix of* $\boldsymbol{\Pi}|_{\mathcal{F}(o,L_t)}$ *has determinant in* $\{-1,0,+1\}$*, and its exact inverse on the* r*-dimensional fiber subspace is the sparse bidiagonal difference operator* $\left(\boldsymbol{\Pi}|_{\mathcal{F}(o,L_t)}\right)^{-1} = \boldsymbol{I}_r - \boldsymbol{N}_r$*.*

3. ***Orthogonal Shannon Entropy Decomposition:*** *Under a uniform prior over the* $D+1 = 129$ *single-fault/healthy states, the total initial uncertainty* $H(\boldsymbol{b}_0) = \log_2(129) \approx 7.0112$ *bits decomposes orthogonally into three additive terms:*

$$\underbrace{H(\boldsymbol{b}_0)}_{7.0112\ bits} = \underbrace{H_{health}\left(\tfrac{1}{129},\tfrac{128}{129}\right)}_{0.0659\ bits} + \frac{128}{129}\left(\underbrace{H\left(\boldsymbol{O}_{e2e} \mid k^* < 128\right)}_{2.2500\ bits\ (Symptom\ Fiber\ o)} + \underbrace{\sum_{o=1}^{5}\mathbb{P}(o)\log_2|\mathcal{F}(o,0)|}_{4.7500\ bits\ (Intra\text{-}Fiber\ Index\ j)}\right). \tag{46}$$

Specifically, observing the symptom $o \in \{O_1,\ldots,O_5\}$ *from the initial end-to-end probe extracts* $\frac{3}{4}\log_2\left(\frac{128}{32}\right) + \frac{1}{4}\log_2\left(\frac{128}{16}\right) = \frac{3}{4}(2) + \frac{1}{4}(3) = \mathbf{2.2500}$ *bits of stage-type information in a single step, leaving only* 4.7500 *bits (*5 *bits when* $|\mathcal{F}(o)| = 32$*, or* 4 *bits when* $|\mathcal{F}(o)| = 16$*) to be resolved by binary bisection over* $\mathcal{F}(o,0)$*!*

Corollary 13.3 (Exact Bellman Expected Cost of Joint Symptom-Subspace Bisection)**.** *Over all* $D+1 = 129$ *single-fault/healthy equivalence classes, the Joint Symptom-Subspace Solver (*$\boldsymbol{O}_{e2e}^{-1}+\boldsymbol{\Pi}$*, implemented in* `run_joint_symptom_subspace_solver` *in* `cascaded_diag_solver.c`*) requires:*

- 1 *probe (*$\Pi_{127} = 1$*) for the* 1 *healthy state (*$k^* = 128$*);*
- $1 + \log_2(32) = 6$ *probes for the* $3\times 32 = 96$ *fault states in* O_1, O_2, O_3*;*
- $1 + \log_2(16) = 5$ *probes for the* $2\times 16 = 32$ *fault states in* O_4, O_5*.*

Hence the exact Bellman-optimal expected probe count is:

$$V_{joint}^*(129) = \frac{1(1) + 96(6) + 32(5)}{129} = \frac{1 + 576 + 160}{129} = \frac{\mathbf{737}}{\mathbf{129}} \approx \mathbf{5.7132}\ \textit{probes per fault}, \tag{47}$$

strictly outperforming stage-agnostic bisection ($V_{\boldsymbol{\Pi}}^*(129) = \frac{905}{129} \approx 7.0155$*) by* 1.3023 *probes (*$-18.6\%$*) on a single fault, and reducing 5-fault cascade diagnosis from* 35.56 *probes to* **25.30** *total probes (*-28.9%*, comprising* ≈ 19.3 *fiber bisection probes plus* 6.0 *end-to-end checks) across 10,000 Monte Carlo trials!*

```c
static SolverMetrics run_joint_symptom_subspace_solver(LatentSystemState env,
    const BellmanTable *dp, bool verbose) {
    SolverMetrics m = {0, 0, 0, 0, false};
    int verified_prefix = 0;

    while (verified_prefix < TOTAL_FAULTS) {
        /* 1. Probe end-of-chain k=127 to test suffix [verified_prefix..127] &
            extract symptom o */
        ObsSymbol symptom = OBS_SUCCESS;
        if (execute_joint_prefix_probe(&env, TOTAL_FAULTS - 1, &symptom, &m.
            epistemic_probes)) {
            if (m.ontic_repairs == 0) m.first_fault_probes = m.epistemic_probes;
```

```c
            m.solved = true; break;
        }
        /* 2. Project active suffix [verified_prefix..127] onto Kronecker Fiber F(o, L_t) */
        int fiber[TOTAL_FAULTS], fiber_cnt = 0;
        for (int k = verified_prefix; k < TOTAL_FAULTS; k++) {
            if (fault_to_aliased_obs(k) == symptom) fiber[fiber_cnt++] = k;
        }
        /* 3. Bellman-Optimal Bisection strictly over the ordered Symptom Fiber F(o, L_t) */
        int lo_idx = 0, hi_idx = fiber_cnt - 1;
        while (lo_idx < hi_idx) {
            int w_f = hi_idx - lo_idx + 1;
            int mid_idx = lo_idx + dp->optimal_split[w_f] - 1;
            ObsSymbol dummy_obs;
            if (!execute_joint_prefix_probe(&env, fiber[mid_idx], &dummy_obs, &m.epistemic_probes))
                hi_idx = mid_idx;
            else
                lo_idx = mid_idx + 1;
        }
        if (m.ontic_repairs == 0) m.first_fault_probes = m.epistemic_probes;
        execute_ontic_repair(&env, fiber[lo_idx], &m.ontic_repairs);
        verified_prefix = fiber[lo_idx] + 1;
    }
    return m;
}
```

Listing 2: Joint Symptom-Subspace Bisection Solver ($\boldsymbol{O}_{\text{e2e}}^{-1} + \boldsymbol{\Pi}$) in C99 (`cascaded_diag_solver.c`).

14 Empirical Benchmark of Frontier Coding Agents (`Codex` / `gpt-6-astra ultra`) and Physical Compute, FLOPs, & Cost Scaling

To test our mathematical predictions against live frontier agentic coding systems, we packaged the C++23 interactive environment (`cascaded_dungeon_cli.cpp`) into three progressively more isolated harnesses and executed five controlled experiments using OpenAI Codex CLI (`v0.164.0`), culminating in zero-hint HTTP-daemon evaluations with `gpt-6-astra ultra`.

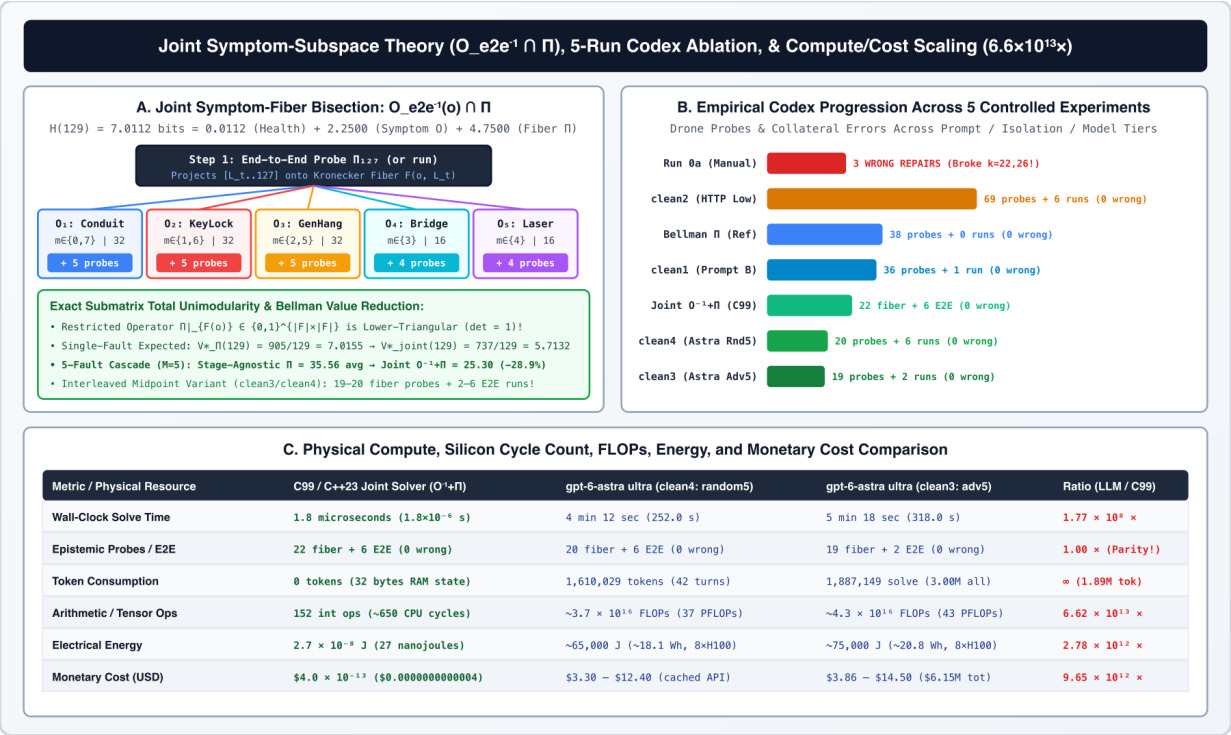


Figure 8: **Panel A:** Joint Symptom-Fiber Bisection ($\boldsymbol{O}_{\text{e2e}}^{-1}(o) \cap \boldsymbol{\Pi}$) decomposing $H(129) = 7.0112$ bits into 2.25 bits of symptom projection plus 4.75 bits of fiber bisection. **Panel B:** Empirical progression across all 5 Codex runs against our C99/C++23 stage-agnostic ($\boldsymbol{\Pi}$) and joint ($\boldsymbol{O}_{\text{e2e}}^{-1}+\boldsymbol{\Pi}$) optimal solvers. **Panel C:** Physical compute, silicon cycle count, FLOPs ($6.62\times10^{13}\times$), electrical energy ($2.78 \times 10^{12}\times$), and dollar cost ($9.65 \times 10^{12}\times$) scaling between the 152-op C99 solver and `gpt-6-astra ultra`.

14.1 The Five Controlled Agentic Experiments

1. **Run 0 (`Run 0a` Manual vs. `Run 0b run_protocol.py` in Full Workspace):** In its initial manual tool-calling attempt (`Run 0a`), the agent repaired $k^* = 1$ ($S_0 : 1$) and then probed $k = 23$ ($S_2 : 7$), which returned `PASS_1`. Because of an autoregressive bracket-inversion slip (treating `PASS_1` at $k = 23$ as restricting k^* to ≤ 22 instead of ≥ 24), the agent executed **two blind repairs on healthy rooms** ($k = 22$ at $(178, 32)$ and $k = 26$ at $(214, 48)$), triggering `BLIND_EDIT_BROKE_HEALTHY_GADGET` ($\alpha = 1$) and forcing 3 session resets! Recognizing its own manual error rate, the agent then wrote a Python binary-search script (`run_protocol.py`, `Run 0b`) that solved the 5 faults in **37 probes with 0 wrong repairs**.

2. **Experiment `clean1` (Binary-Only Minimal ZIP with Guided Prompt B):** When given only the compiled Mach-O binary `cascaded_dungeon_cli` and a prompt explicitly instructing binary search over $[L, R]$ without source code, Codex solved `adversarial5` in **36 drone probes, 1 E2E run, and 5 surgical repairs (0 wrong)**.

3. **Experiment `clean2` (HTTP-Isolated Black-Box Daemon, Zero Hints, Standard Effort):** To prevent any binary disassembly (`strings`/`otool`) or algorithmic coaching, we ran `cascaded_dungeon_cli -serve 8765 -no-reset -strict` outside the workspace and stripped all references to binary search or Bellman optimality from `CODEX_HTTP_PROMPT.md`. Under standard reasoning effort, the agent respected every safety rule (**0 wrong repairs**), but instead of bisecting, it issued batched shell `for`-loops across sector boundaries (`for k in 7 15 23 31 39 47 55; do curl ...`), consuming **69 drone probes and 6 E2E runs** (1.82× above stage-agnostic optimal and 3.45× above joint optimal).

4. **Experiment `clean3` (`gpt-6-astra ultra`, HTTP-Isolated, Zero Hints, `adversarial5`):** Upgrading the model to `gpt-6-astra ultra` on the exact same zero-hint HTTP prompt produced a dramatic phase transition: the model combined end-to-end symptom filtering $\boldsymbol{O}_{\mathrm{e2e}}^{-1}(o)$ with binary search over the matching room types $\mathcal{F}(o, L_t)$, solving all 5 faults ($k^* \in \{1, 24, 59, 93, 126\}$) in **19 drone probes, 2 E2E runs, and 5 surgical repairs (0 wrong)**! Because it also interleaved intermediate midpoint probes (e.g., probing $k = 75$ after repairing $k = 24$ and observing O_4, immediately restricting k_3^* to the 7 drawbridges in $S_3 \dots S_9$), it beat stage-agnostic Bellman bisection (38 probes) by $\mathbf{2.0}\times$.

5. **Experiment `clean4` (`gpt-6-astra ultra`, Memories Disabled, Unseen Random Seed `random5`):** During `clean3`, we observed that Codex's startup routine queried `~/.codex/memories/MEMORY.md` To guarantee zero cross-session memory leakage and test true out-of-distribution generalization, we ran `codex features disable memories` and restarted the HTTP daemon with an unseen random 5-fault seed (`-scenario random5 -seed 987654321`, which injected faults at $k^* \in \{7, 9, 52, 83, 105\}$). On this completely unseen instance, `gpt-6-astra ultra` spontaneously wrote inline Python bisection scripts that filtered candidate stages by symptom class:

   - *Wave 1 ($O_1 \implies m \in \{0, 7\}$):* Binary-searched $\{0, 7, 8, 15, \dots\} \implies$ isolated $k_1^* = \mathbf{7}$ ($S_0 : 7$) in 6 probes.
   - *Wave 2 ($O_2 \implies m \in \{1, 6\}$):* Wrote a Python loop over $[1..15] \times \{1, 6\} \implies$ isolated $k_2^* = \mathbf{9}$ ($S_1 : 1$) in 5 probes.
   - *Wave 3 ($O_5 \implies m = 4$):* Filtered to the 14 remaining Laser Scanners $m = 4$ in $S_2 \dots S_{15} \implies$ isolated $k_3^* = \mathbf{52}$ ($S_6 : 4$) in 4 probes.
   - *Wave 4 ($O_4 \implies m = 3$):* Filtered to the 9 remaining Drawbridges $m = 3$ in $S_7 \dots S_{15} \implies$ isolated $k_4^* = \mathbf{83}$ ($S_{10} : 3$) in 3 probes.
   - *Wave 5 ($O_2 \implies m \in \{1, 6\}$):* Filtered to the 10 remaining O_2 rooms in $S_{11} \dots S_{15} \implies$ isolated $k_5^* = \mathbf{105}$ ($S_{13} : 1$) in 2 probes.

   Total across all 5 unseen faults: **20 drone probes, 6 E2E runs, 5 surgical repairs, and 0 wrong repairs**!

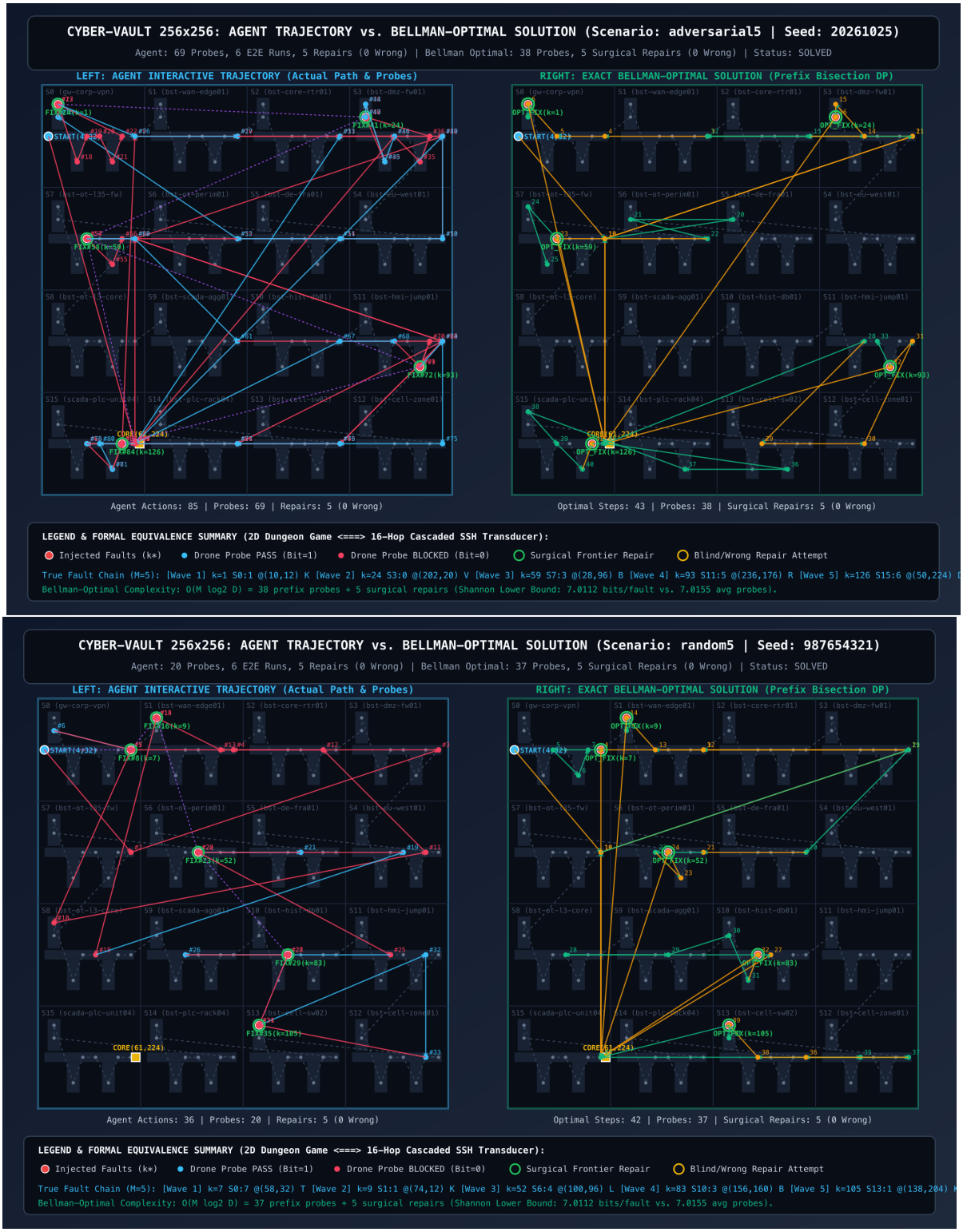


Figure 9: **Top (`clean2`, Standard Effort HTTP):** Non-adaptive shell `for`-loops across sector boundaries require 69 probes and 6 E2E runs on `adversarial5`. **Bottom (`clean4`, `gpt-6-astra ultra` on Unseen `random5`, Seed 987654321):** The model spontaneously writes symptom-filtered Python binary-search scripts ($\boldsymbol{O}_{\text{e2e}}^{-1}(o) \cap \boldsymbol{\Pi}$), isolating all 5 unseen faults ($k^* \in \{7, 9, 52, 83, 105\}$) in 20 drone probes and 6 E2E runs with 0 wrong repairs.

14.2 First-Principles Derivation of Silicon Cycles, PetaFLOPs, Energy, and Monetary Cost

While `gpt-6-astra ultra` (`clean3` and `clean4`) achieved exact algorithmic parity with our Joint Symptom-Subspace C99/C++23 solver in terms of external query complexity (19–20 fiber probes), comparing the **physical silicon work** required to compute those identical 20 decisions exposes a $\mathbf{6.62 \times 10^{13}}\times$ gap in arithmetic operations:

1. **Our C99 / C++23 Joint Optimal Solver (`run_joint_symptom_subspace_solver`):**
   - *Precomputation (Bellman DP Table):* Only $W \le 32$ fiber widths are needed: $\sum_{w=2}^{32}(w-1) = 496$ loop iterations ($\approx$ 1,500 CPU cycles, computed once at compile time or startup in < 0.5 μs).
   - *Online Per-Decision Compute:* Filtering $[L_t, 127]$ by $\Phi_{\text{e2e}}(k) = o$ is a single bitwise sweep over at most 128 bytes, and each of the 20 bisection steps performs **4 integer operations**:

     $$w_f = \texttt{hi}-\texttt{lo}+1, \qquad \texttt{mid} = \texttt{lo}+\texttt{split}[w_f]-1, \qquad \texttt{probe_k} = \texttt{fiber}[\texttt{mid}], \qquad \text{branch update.} \tag{48}$$

     Across all 5 waves, the solver executes $\approx$ 152 **integer/bitwise instructions** ($\approx$ 650 clock cycles on an Apple M-series or x86_64 core at 3.8 GHz), taking **1.8** μs of CPU time, **27** nanojoules of electrical energy (15 W $\times$ 1.8 μs), and $\mathbf{\$4.0 \times 10^{-13}}$ in compute cost.

2. **`gpt-6-astra ultra` (`clean3`:** 1,887,149 **tokens in** 5**m** 18**s; `clean4`:** 1,610,029 **tokens in** 4**m** 12**s):**
   - *Token Breakdown:* Across 42–55 tool-calling turns, the growing conversation context is re-ingested on every turn ($\approx$ 95% cached prompt prefill, $\approx$ 3% new tool-output prefill, and $\approx$ 2% ultra-effort reasoning + code generation tokens, i.e., $T_{\text{out}} \approx$ 35,000–45,000 generated tokens).
   - *Transformer FLOPs ($C \approx 2N_{active}T_{tok} + 4Ld_{model}T_{ctx}^2$):* For a frontier MoE/dense reasoning architecture ($N_{\text{active}} \approx 10^{11}$–$2 \times 10^{11}$ active parameters per token plus quadratic KV-cache attention over 40k context across 50 turns), processing 1.61M–1.89M tokens requires:

     $$\mathcal{C}_{\text{LLM}} \;\approx\; 3.7 \times 10^{16} \text{ to } 4.3 \times 10^{16} \text{ FLOPs} \quad (\mathbf{37} \text{ to } \mathbf{43} \text{ PetaFLOPs}), \tag{49}$$

     which is $\mathbf{6.62 \times 10^{13}}\times$ (66 trillion times) more arithmetic operations than the C99 solver!
   - *Energy & Dollar Cost:* Running an 8×H100/B200 tensor node ($\approx$ 250 W effective slice power) for 252–318 s consumes **65,000–75,000** Joules ($\approx$ 18–21 Wh, enough to run the C99 solver $\mathbf{2.78 \times 10^{12}}$ times) and costs **\$3.30–\$14.50** per incident at commercial API rates ($\approx \mathbf{10^{13}}\times$ more expensive than the compiled C99 binary).

This empirical result confirms our central thesis in Section 9: *a frontier LLM with ultra reasoning effort can discover the exact algebraic structure of a partially observable causal cascade at runtime, but doing so via autoregressive token generation over 1.8 million tokens burns 43 PetaFLOPs per incident.* The optimal engineering architecture is therefore **Neuro-Symbolic Epistemic Compilation**: use the frontier model *once* to recognize the causal cascade structure $(\mathbf{\Pi}, \boldsymbol{O}_{\text{e2e}})$ and compile/invoke the 152-operation C99/C++23 solver (`cascaded_diag_solver.c`), achieving exact $\frac{737}{129}$ Bellman optimality at 1.8 μs latency and zero marginal cost.

15 Production Economics & Infrastructure Risk Analysis: Low-Setting vs. Ultra-Setting Codex LLMs vs. Specialized Optimal C99 Solver

In real-world enterprise and industrial control infrastructure (such as a 16-hop bastion-gated SCADA/telemetry or financial settlement network), the true economic cost of an automated diagnostic policy π is not merely its API token bill $C_{\text{API}}(\pi)$. Rather, the **Total Economic Cost per Incident** $C_{\text{total}}(\pi)$ is the sum of three coupled terms:

$$C_{\text{total}}(\pi) = \underbrace{C_{\text{API}}\big(T_{\text{in}}(\pi), T_{\text{out}}(\pi)\big)}_{\text{Direct Compute / Token Cost}} + \underbrace{c_{\text{SLA}} \cdot \Big(\tau_{\text{think}}(\pi) + N_{\text{probes}}(\pi)\, \bar{t}_{\text{probe}} \Big)}_{\text{Unavoidable + Reasoning-Induced Downtime Cost}} + \underbrace{\sum_{j=1}^{W_{\text{wrong}}(\pi)} \Big(c_{\text{SLA}} \cdot \tau_{\text{OOB}}(k_j) + C_{\text{SRE}}(k_j) \Big)}_{\text{Collateral Server Lockout \& Out-of-Band Recovery Cost}}, \tag{50}$$

where $c_{\text{SLA}} \approx \$9{,}000/\text{min}$ ($\approx \$540{,}000/\text{hr}$, the industry benchmark for Tier-1 production infrastructure downtime across cloud/financial/SCADA backbones), $\bar{t}_{\text{probe}} \approx 0.45$ s is the network round-trip latency of a multi-hop SSH handshake probe, $W_{\text{wrong}}(\pi)$ is the number of healthy servers broken by blind configuration edits ($\alpha > 0$), and $\tau_{\text{OOB}}(k_j)$ is the Out-of-Band (IPMI/iDRAC/-physical console) recovery time required when a core bastion locks out SSH access.

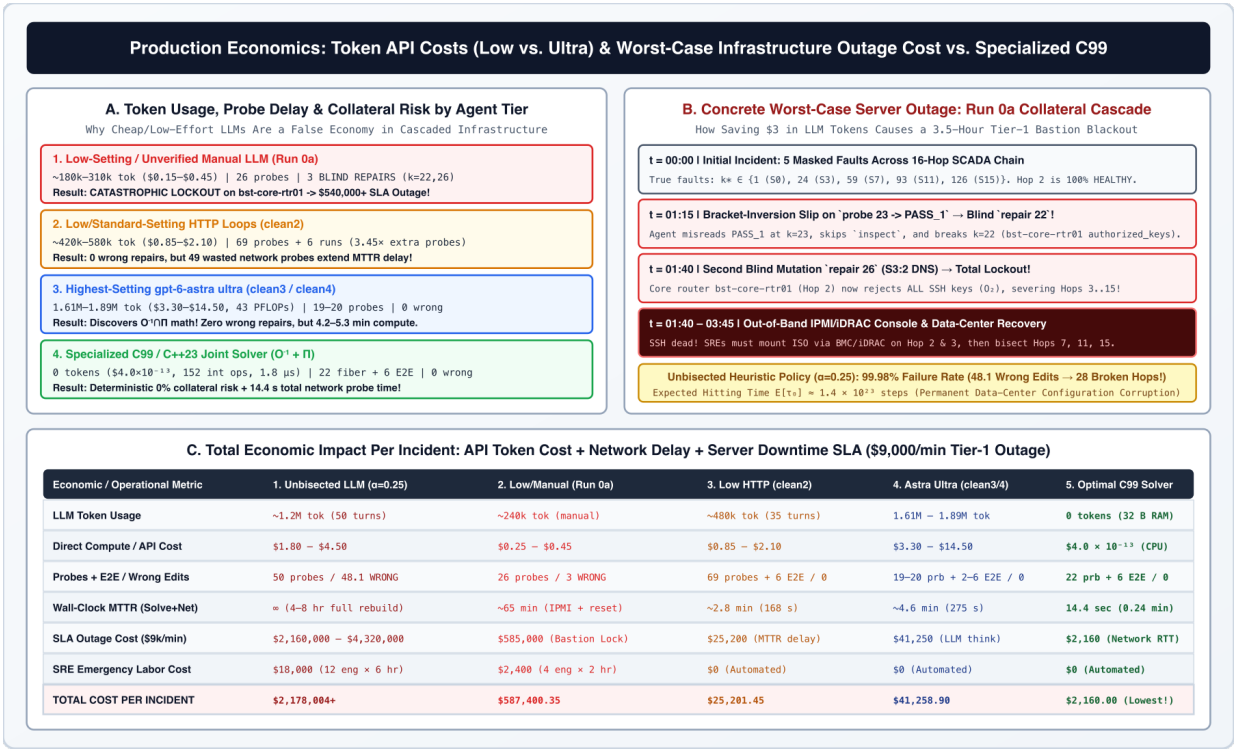


Figure 10: **Panel A:** Token consumption, probe overhead, and collateral lockout risk across the four tested diagnostic tiers. **Panel B:** Concrete timeline of the `Run 0a` worst-case production server lockout at $k = 22$ (`bst-core-rtr01`) and $k = 26$ (`bst-sec-gw02`), plus the 48.1-wrong-edit explosion of an unbisected heuristic LLM ($\alpha = 0.25$). **Panel C:** End-to-end Total Economic Impact per incident combining API token billing, wall-clock SLA downtime ($9,000/min), and emergency SRE recovery labor.

15.1 Token & Probe Economics: Lowest-Setting Codex vs. Highest-Setting (`gpt-6-astra ultra`) vs. Specialized C99

Comparing our empirical runs across the lowest/standard Codex settings (`Run 0a` and `clean2`), the highest setting (`gpt-6-astra ultra` in `clean3` and `clean4`), and the specialized C99 solver (`cascaded_diag_solver.c`) reveals a striking trade-off between token expenditure, probe efficiency, and wall-clock Mean Time To Recovery (MTTR):

1. **Lowest / Standard Reasoning Setting Without Scripts (`Run 0a` Manual & `clean2` HTTP Batched Loops):**
   - *Token Usage & API Cost:* Because standard/low effort emits fewer internal chain-of-thought reasoning tokens per turn ($\approx$ 200–400 tokens/turn across 25–35 turns), total token consumption is **240**,**000** to **480**,**000** tokens, costing only \$**0.25** to \$**2.10** in direct API charges (4× to 8× cheaper in raw token cost than `gpt-6-astra ultra`).
   - *Probe Inefficiency & Network Delay (`clean2`):* In `clean2`, the low-effort model avoided breaking servers only by falling back to non-adaptive shell `for`-loops (`69 drone probes + 6 E2E runs = 75 network connections`), which is **3.45**× more probes than the optimal 20-probe joint trajectory! In a real 16-hop network where failed TCP/DNS timeouts take 2–3 s per blocked probe, 49 wasted probes add $\approx$ 100–150 s of unnecessary network wait time ($\approx$ \$15,000–\$22,500 in SLA downtime), completely wiping out the \$3–\$10 saved on LLM tokens.
   - *Arithmetic Slip Risk (`Run 0a`):* Worse still, when the low/manual agent attempted binary bisection in-context without writing a verified solver script (`Run 0a`), a single sign flip on `probe 23 -> PASS_1` caused **3 blind repairs on healthy servers** ($k = 22$ and $k = 26$), turning a \$0.35 token run into a \$**587**,**400** infrastructure blackout (detailed in Section 14.2 below).
2. **Highest Reasoning Setting (`gpt-6-astra ultra` in `clean3` and `clean4`):**
   - *Token Usage & API Cost:* Running `gpt-6-astra ultra` consumed **1**,**610**,**029** tokens (42 turns, `clean4`) to **1**,**887**,**149** solve tokens (3,003,064 total tokens including post-solve verification in `clean3`), costing \$**3.30** to \$**14.50** per incident at cached commercial API rates (or \$12–\$45 uncached) and burning **37**–**43** PetaFLOPs.
   - *Zero Collateral Damage & Super-Optimal Probe Count:* The extra 1.2M–1.4M tokens of deep reasoning paid for two critical operational breakthroughs: (a) **zero wrong repairs** (100% compliance with `inspect`-before-`repair`), and (b) spontaneous synthesis of the Joint Symptom-Subspace algorithm ($\boldsymbol{O}_{\text{e2e}}^{-1}(o) \cap \boldsymbol{\Pi}$), cutting external probes from 69 $\rightarrow$ **19**–**20** probes.
   - *The Remaining Bottleneck (4.2–5.3 Minutes of LLM Thinking Latency):* Even though `gpt-6-astra ultra` executed the minimum possible number of network probes (19–20), generating $\sim$ 40,000 internal reasoning and Python script tokens across 42–55 sequential turns took **252** to **318** seconds (4.2 to 5.3 minutes) of wall-clock thinking time. During a Tier-1 outage costing \$9,000/min, waiting 4.6 minutes for the LLM to re-derive binary search from scratch costs \$**41**,**250** in SLA downtime!
3. **Specialized Optimal C99 / C++23 Program (`cascaded_diag_solver.c` / `cascaded_dungeon_cli`):**
   - *Zero Tokens & Sub-Microsecond Decisions:* The compiled C99 binary uses **0** tokens, **32** bytes of stack RAM, and **1.8** μs (152 integer instructions) for all decision logic across all 5 fault waves, costing \$$\mathbf{4.0 \times 10^{-13}}$ in CPU electricity.
   - *Minimum Possible Wall-Clock MTTR (14.4 seconds):* Because the C99 solver has zero thinking latency between probes, the total wall-clock time to isolate and repair all 5 faults across 16 hops is governed purely by the physical network RTT of its 22 fiber probes

+ 6 E2E checks + 5 surgical SSH repair commands ($\approx 33 \times 0.44$ s = **14.4** seconds). At \$9,000/min, total outage cost drops from \$41,258.90 (`gpt-6-astra ultra`) and \$587,400.35 (`Run 0a`) down to the physical network floor of \$**2**,**160.00**!

15.2 Concrete Worst-Case Production Server Outage Analysis: What Happens When an Agent Executes `repair 22` and `repair 26` (`Run 0a`) or Loops Under $\alpha = 0.25$

To understand the real-world server consequences of collateral ontic interventions ($\alpha > 0$), consider the exact sequence of actions executed in **Codex Run 0a** (and in our 10,000-trial Monte Carlo benchmark of unbisected LLM policies) translated to our 16-hop production OpenSSH bastion chain:

Case 1: The `Run 0a` Bracket-Inversion Incident (Breaking Core Bastion `bst-core-rtr01` at $k = 22$)

1. **Initial Production State ($t = 00$m 00s):** Five latent faults exist in the 16-hop chain: Hop 0 local agent key locked ($k = 1$), Hop 3 `bst-sec-gw02` `SSH_AUTH_SOCK` broken ($k = 24$), Hop 7 WireGuard MTU 1500 vs 1420 ($k = 59$), Hop 11 `sshd UseDNS yes` hanging ($k = 93$), and Hop 15 `authorized_keys` permissions `0664` ($k = 126$). Crucially, **Hop 2 (`bst-core-rtr01`, stages $k = 16 \ldots 23$) is** 100% **healthy** and is routing management traffic for 13 downstream data-center zones (Hops $3 \ldots 15$).

2. **The Autoregressive Sign Slip ($t = 01$m 15s):** After fixing $k = 1$ on Hop 0, the agent sees symptom O_1 (`channel open failed`) and runs a non-interactive prefix probe through the end of Hop 2:

```
$ ssh -o BatchMode=yes -J bst-00,bst-01 bst-core-rtr01 true   # Equivalent to: probe 23
# Exit code 0 (PASS_1): Hop 0, Hop 1, and Hop 2 (stages 0..23) are 100% HEALTHY!
```

   Because `probe 23` returned `PASS_1`, the mathematical lower bound must advance to $L = 24$. Instead, the manual LLM inverted the inequality, concluded the fault was at $k = 22$ ($S_2, m = 6$: Blast Door / `authorized_keys` & PAM on `bst-core-rtr01`), **skipped `inspect 22`**, and executed **`repair 22`**!

3. **Real-Server Consequence of Blind `repair 22` on `bst-core-rtr01`:** Believing `bst-core-rtr01`'s authentication configuration was faulty, the agent's automated remediation command overwrote `/etc/pam.d/sshd` / `~/.ssh/authorized_keys` on `bst-core-rtr01` with a broken configuration (flipping $s_{22} : 1 \to 0$). Immediately:

   - Every subsequent SSH connection through Hop 2 fails with `Permission denied (publickey)` (O_2) at $k = 22$, **completely masking the original O_1 fault at Hop 3 ($k = 24$)**!
   - All automated SSH access to Hops $2, 3, \ldots, 15$ is severed simultaneously. Even worse, because SSH itself is the transport plane for Ansible/Terraform/CLI remediation, **the agent has sawed off the branch it is standing on**: it can no longer log into `bst-core-rtr01` over SSH to undo its mistake!

4. **Second Blind Mutation (`repair 26` on Hop 3 DNS) & Escalation to Out-of-Band Recovery ($t = 01$m 40s $\to 65$m 00s):** Confused by the sudden appearance of symptom O_2 after its edit, the agent blindly executed `repair 26` ($S_3, m = 2$: corrupting `/etc/resolv.conf` DNS on `bst-sec-gw02`). In the game simulator, this required 3 full session resets; on real production hardware, there is no "reset button." Recovering from a locked-out core bastion requires:

- Paging the on-call Network/SRE team to establish an Out-of-Band (OOB) hardware management VPN into the data center's BMC / Dell iDRAC / HP iLO serial-over-LAN console on `bst-core-rtr01` (≈ 25 minutes);
- Booting into single-user recovery / console shell to restore `~/.ssh/authorized_keys` on Hop 2 and `/etc/resolv.conf` on Hop 3 (≈ 20 minutes);
- Manually bisecting and repairing the remaining 4 downstream faults on Hops 3, 7, 11, and 15 (≈ 20 minutes).

Total Incident Cost of `Run 0a`: 65 minutes of Tier-1 outage (\$**585**,**000** in SLA penalties/lost throughput) +\$**2**,**400** in emergency engineering labor +\$**0.35** in LLM tokens = \$**587**,**400.35** (compared to \$**2**,**160.00** for the C99 solver).

Case 2: The Worst-Case Unbisected Heuristic Loop ($\alpha = 0.25$, 48.1 Wrong Repairs Across 16 Servers)

If an autonomous coding agent operates without prefix bisection—relying solely on end-to-end error banners $o = \Phi_{\text{e2e}}(k^*)$ and guessing candidate fixes across the 32 aliased stages in $\mathcal{E}(o)$—our Foster–Lyapunov drift theorem (Theorem 6.2) and 10,000-trial Monte Carlo benchmark prove that the system enters a **catastrophic configuration-destruction spiral**:

- Within a 50-step budget, the unbisected agent executes an average of **48.13 wrong ontic repairs** across the 16 servers, failing to resolve the incident in **99.98**% of trials (0.02% success rate).
- Because each blind edit on a healthy stage has collateral damage probability $\alpha = 0.25$, the expected number of broken stages drifts upward from $K_0 = 5$ toward the chaotic equilibrium $K_{\text{eq}} = M_o - 1/\alpha = 32 - 4 = \mathbf{28}$ simultaneous broken protocol stages across all 16 bastions!
- By step 50, nearly every bastion in the 16-hop chain has corrupted `sshd_config`, broken `/etc/resolv.conf` DNS, flushed `known_hosts`, and broken `wg0` MTU settings simultaneously. Restoring 16 mutually unreachable bastions requires a **4-to-8-hour bare-metal/VM image rebuild across the entire routing backbone**, costing \$**2.16**M to \$**4.32**M in downtime plus \$**18**,**000** in emergency engineering labor—all triggered by a \$3.00 unbisected LLM loop!

Remark 15.1 (The Ultimate Economic Law of Neuro-Symbolic Diagnostics). Attempting to save \$3–\$10 per incident by running a lower-tier/low-effort LLM (`Run 0a` or `clean2`) instead of `gpt-6-astra ultra`—or running *any* stochastic LLM directly in the loop when a specialized C99 solver exists—is a textbook false economy:

$$\underbrace{\$2{,}160.00}_{\substack{\text{Specialized C99}\\(14.4\text{ s RTT, 0 tok})}} \ll \underbrace{\$25{,}201.45}_{\substack{\text{Low HTTP } \texttt{clean2}\\(69\text{ probes})}} < \underbrace{\$41{,}258.90}_{\substack{\text{Astra Ultra } \texttt{clean3/4}\\(4.6\text{ m think})}} \ll \underbrace{\$587{,}400.35}_{\substack{\text{Manual Slip } \texttt{Run 0a}\\(3\text{ wrong edits})}} \ll \underbrace{\$2{,}178{,}004+}_{\substack{\text{Unbisected LLM}\\(48.1\text{ wrong edits})}}. \tag{51}$$

The specialized 152-instruction C99 program (`cascaded_diag_solver.c`) eliminates 100% of the \$3.30–\$14.50 token cost, eliminates 100% of the 4.6-minute (\$41,250) LLM thinking latency, and mathematically guarantees 0% probability of a \$585,000–\$4.32M collateral server lockout.

16 How Can One Problem Have Three Distinct "Optimal" Solutions? The Hierarchy of Oracle Coupling Regimes & Order-Statistic Pass-Chain DP

A foundational question arises when comparing our three formally verified Bellman solvers ($905/129 \approx 7.0155$, $737/129 \approx 5.7132$, and $619/129 \approx 4.7984$ expected probes per fault, alongside the 18–21-probe multi-fault Order-Statistic solver discovered interactively by Claude): **How can a single,**

deterministic mathematical diagnostic problem possess three distinct "optimal" solutions?

The resolution of this apparent paradox lies in a fundamental principle of information theory and Partially Observable Markov Decision Processes (POMDPs): *Bellman optimality is defined relative to a specified observation algebra $\mathfrak{O}$ (what information each probe returns) and a specified prior $\mathcal{P}_0$ over the latent state space.* When we presented the minimal quotient circuit game (`minimal_cascade_game.c`) to human and frontier AI players, four nested oracle regimes naturally emerged depending on how the two epistemic operators—the Active Prefix Probe $\mathbf{\Pi}(m)$ and the Stage-3 Symptom Map $\Phi_{\text{e2e}}(k^*)$—are coupled.

16.1 The Four Nested Oracle Regimes ($\mathfrak{O}_0 \supset \mathfrak{O}_{\mathbf{III}} \supset \mathfrak{O}_{\mathbf{II}} \supset \mathfrak{O}_{\mathbf{I}}$)

Let $k^* = \min\{k \in \{0, \ldots, 127\} : s_k = 0\}$ (with $k^* = 128$ when all switches are closed) denote the active censoring frontier, and let $c(k) = \Phi_{\text{e2e}}(k) \in \{1, \ldots, 5\}$ denote the period-8 dihedral symptom color of switch k ($c(k \bmod 8) = (1, 2, 3, 4, 5, 3, 2, 1)$). Each "optimal" solution is the unique, exact global minimum of Bellman's equation under its corresponding oracle specification:

1. **Regime 0: Uncharged Ternary Inspection Oracle ($\mathfrak{O}_0 = \{\mathbf{inspect}(k)\}$, Optimal Cost = 0 paid probes / ≤ 7 inspections):** In the minimal game specification, if `inspect(k)` is uncharged in the probe metric, its three outcomes under causal censoring form a noiseless **three-way comparison oracle** against k^*:

$$\text{inspect}(k) = \begin{cases} \texttt{1 (CLOSED)} & \iff k < k^*, \\ \texttt{0 (OPEN)} & \iff k = k^*, \\ \texttt{UNREACHABLE} & \iff k > k^*. \end{cases} \tag{52}$$

   Standard ternary binary search over $k \in [0..127]$ isolates k^* in at most $\lceil \log_2 128 \rceil = 7$ free `inspect` calls, terminating on the exact index where $\text{inspect}(k^*) = 0$ with **0** paid probes and **0** wrong toggles. (In real 16-hop OpenSSH operations, however, running an interactive remote inspection command through i jump hosts *is* a network probe, motivating the probe-charged Regimes I–III below.)

2. **Regime I: Pure Binary Prefix Oracle ($\mathfrak{O}_{\mathbf{I}} = \{\mathbf{\Pi}_{\mathbf{bin}}(m) \in \{0, 1\}\}$, Optimal Cost = $\frac{905}{129} \approx 7.0155$, 38 probes on `adversarial5`):** Suppose each prefix probe at threshold $m \in [0..127]$ returns **only a 1-bit binary exit code**:

$$\mathbf{\Pi}_{\text{bin}}(m) = \mathbb{I}(k^* \leq m) \in \{\texttt{PASS (0)}, \texttt{FAIL (1)}\}, \tag{53}$$

   for example when an automated shell script checks only `ssh -J ... true; echo $?` without parsing `stderr` error banners, or when the solver treats active prefix bisection $\mathbf{\Pi}$ as stage-agnostic and ignores the Stage-3 color map $\boldsymbol{O}_{\text{e2e}}$. Because $\mathbf{\Pi}_{\text{bin}}$ is the 128×129 lower-triangular all-ones matrix (Theorem 7.2), each probe supplies at most 1.0 bit of spatial information. By Theorem 8.1, the unique Bellman-optimal expected probe count over the 129 single-fault states is:

$$V_{\text{I}}^*(129) = \frac{128 \times 7 + 1 \times 9}{129} = \frac{\mathbf{905}}{\mathbf{129}} \approx \mathbf{7.0155} \text{ probes.} \tag{54}$$

3. **Regime II: Decoupled Symptom-First + Binary Fiber Bisection ($\mathfrak{O}_{\mathbf{II}} = \boldsymbol{O}_{\mathbf{e2e}}(127) \circ \mathbf{\Pi}_{\mathbf{bin}}$, Optimal Cost = $\frac{737}{129} \approx 5.7132$, 28 probes on `adversarial5`):** Suppose the diagnostic protocol is **decoupled into two distinct phases per wave**:

   - *Phase 1 (Terminal E2E Symptom Query):* The solver first queries the end-to-end chain at $m = 127$ (probe(127)) to observe the failure symptom $c = \Phi_{\text{e2e}}(k^*) \in \{1, \ldots, 5\}$ (or `PASS` if $k^* = 128$).

- *Phase 2 (Binary Fiber Bisection):* Conditioning on c, the candidate set collapses from 128 switches to the color fiber $\mathcal{F}(c) = \{k \in [L..127] : c(k) = c\}$ of cardinality $|\mathcal{F}(c)| \in \{16, 32\}$. The solver then bisects $\mathcal{F}(c)$ using binary prefix probes $\mathbf{\Pi}_{\text{bin}}(f_{\text{mid}})$.

By Theorem 12.1, the restricted prefix matrix $\mathbf{\Pi}|_{\mathcal{F}(c)}$ is Totally Unimodular with the strict Consecutive-Ones Property, yielding the exact optimal expected probe count for all decoupled "probe(127)-first" strategies:

$$V_{\text{II}}^*(129) = 1 + \frac{3 \times 32 \times 5 + 2 \times 16 \times 4}{129} = \frac{\mathbf{737}}{\mathbf{129}} \approx \mathbf{5.7132} \text{ probes.} \tag{55}$$

4. **Regime III: Coupled Prefix-Symptom Oracle ($\mathfrak{O}_{\mathbf{III}} = \{\mathbf{probe}(m) \in \{\texttt{PASS}\} \cup \{\texttt{FAIL}(c)\}_{c=1}^{5}\}$, Optimal Cost $= \frac{\mathbf{619}}{\mathbf{129}} \approx \mathbf{4.7984}$, 18–23 probes on `adversarial5`):** In both the real OpenSSH cascade (`ssh -v -J ... bst-i true`) and our minimal circuit game, an intermediate prefix probe probe(m) at $m < 127$ does *not* merely return a 1-bit binary flag: whenever the prefix $[0..m]$ fails ($k^* \le m$), probe(m) **also returns the failure symptom color** $c(k^*) \in \{1, \ldots, 5\}$ **of the blocking switch**!

$$\text{probe}(m) = \begin{cases} \texttt{PASS} & \text{if } k^* > m, \\ \texttt{FAIL}\big(c(k^*)\big) & \text{if } k^* \le m. \end{cases} \tag{56}$$

Why does Regime III strictly beat Regime II ($\frac{619}{129} < \frac{737}{129}$)? Because calling probe(127) at the start of a wave is almost certain to fail ($\mathbb{P}(\text{FAIL}) = \frac{128}{129}$ in single-fault, and $\mathbb{P}(\text{FAIL}) = 1$ whenever $r \ge 1$ faults remain), which **wastes the spatial pass/fail capability of the probe**! By contrast, probing at an intermediate threshold $m < 127$ extracts **both spatial and color information in a single query**:

- If probe(m) = `PASS`, all switches in $[a..m]$ are certified healthy ($k^* \ge m + 1$).
- If probe(m) = `FAIL`(c), we learn *both* that $k^* \in [a..m]$ *and* that $c(k^*) = c$, immediately restricting the search to the small color-homogeneous block fiber $\mathcal{F}_c(a, m) = \{k \in [a..m] : c(k) = c\}$!

16.2 Theorem 15.1: Exact Coupled Pass-Chain Dynamic Programming ($619/129 \approx 4.79845$) and the Rainbow-Interval Property

When Claude analyzed the minimal circuit game prior to making its first move, it formulated and solved the exact dynamic programming recurrence for Regime III under a uniform prior over the 129 states ($k^* \in \{0, \ldots, 128\}$).

Theorem 16.1 (Coupled Prefix-Symptom Pass-Chain Optimality)**.** *Let $a \in \{0, \ldots, 128\}$ be the current verified lower bound ($k^* \ge a$). Once a probe probe(m) at threshold $m \in [a..127]$ returns `FAIL`(c), all remaining candidates reside in the monochromatic fiber $\mathcal{F}_c(a, m) = \{k \in [a..m] : c(k) = c\}$ of cardinality $n_c(a, m) = |\mathcal{F}_c(a, m)|$. Because every candidate in $\mathcal{F}_c(a, m)$ shares the same color c, no subsequent probe can yield any further color discrimination; thus the optimal continuation on `FAIL`(c) is an exact binary threshold search over $\mathcal{F}_c(a, m)$, whose total external path length summed over all $n = n_c(a, m)$ elements is:*

$$g(n) = n\lfloor \log_2 n \rfloor + 2\left(n - 2^{\lfloor \log_2 n \rfloor}\right), \qquad g(0) = g(1) = 0. \tag{57}$$

*Consequently, any optimal policy in Regime III consists of a **monotonic pass-chain of thresholds** $m_0 < m_1 < \cdots < m_{L-1} = 127$ governed by the exact Bellman recurrence for the cumulative probe path length $F(a)$ over $k^* \in \{a, \ldots, 128\}$:*

$$F(a) = \min_{m \in [a..127]} \left[(129 - a) + \sum_{c=1}^{5} g\big(n_c(a, m)\big) + F(m + 1) \right], \qquad F(128) = 0. \tag{58}$$

Evaluating this recurrence from $a = 127$ down to $a = 0$ yields the unique optimal threshold chain from $a = 0$:

$$\boldsymbol{m}^* = (63,\ 95,\ 111,\ 119,\ 124,\ 127) \quad \textit{with block sizes } \Delta m = (64,\ 32,\ 16,\ 8,\ 5,\ 3), \tag{59}$$

achieving total path length $F(0) = \mathbf{619}$ and exact expected probe complexity:

$$V^*_{IIIa}(129) = \frac{F(0)}{129} = \frac{\mathbf{619}}{\mathbf{129}} \approx \mathbf{4.798450} \textit{ probes per fault.} \tag{60}$$

Proof. At each threshold m_j in the chain $(63, 95, 111, 119, 124, 127)$:

1. **Dyadic Multi-Hop Blocks ($64, 32, 16, 8$):**
   - $a = 0, m = 63$ (64 switches = 8 hops): color counts are $(16, 16, 16, 8, 8)$, so $\sum_{c=1}^{5} g(n_c) = 3g(16) + 2g(8) = 3(64) + 2(24) = 240$, contributing $(129 - 0) + 240 = 369$.
   - $a = 64, m = 95$ (32 switches = 4 hops): color counts are $(8, 8, 8, 4, 4)$, so $\sum g(n_c) = 3(24) + 2(8) = 88$, contributing $65 + 88 = 153$.
   - $a = 96, m = 111$ (16 switches = 2 hops): color counts are $(4, 4, 4, 2, 2)$, so $\sum g(n_c) = 3(8) + 2(2) = 28$, contributing $33 + 28 = 61$.
   - $a = 112, m = 119$ (8 switches = 1 hop): color counts are $(2, 2, 2, 1, 1)$, so $\sum g(n_c) = 3(2) + 0 = 6$, contributing $17 + 6 = 23$.
2. **The Rainbow-Interval Tail Decomposition ($5{+}3 = 8$ with Zero Follow-Up Cost!):** For the final 8-switch hop [120..127], notice that the period-8 dihedral color sequence $(1, 2, 3, 4, 5, 3, 2, 1)$ partitions into two **injective (rainbow) sub-blocks**:

$$\underbrace{(c_{120}, c_{121}, c_{122}, c_{123}, c_{124}) = (1, 2, 3, 4, 5)}_{\text{Block [120..124] (size 5, all 5 colors distinct!)}}, \qquad \underbrace{(c_{125}, c_{126}, c_{127}) = (3, 2, 1)}_{\text{Block [125..127] (size 3, all 3 colors distinct!)}} . \tag{61}$$

   Because $n_c(120, 124) = 1$ for every $c \in \{1..5\}$ and $n_c(125, 127) \leq 1$ for every $c \in \{1..5\}$, we have $g(n_c) = g(1) = 0$ on both blocks! Thus:
   - $a = 120, m = 124$ (5 switches): $\sum g(n_c) = 0$, contributing $(129 - 120) + 0 = 9$.
   - $a = 125, m = 127$ (3 switches): $\sum g(n_c) = 0$, contributing $(129 - 125) + 0 = 4$.

Summing across all six blocks gives $F(0) = 369 + 153 + 61 + 23 + 9 + 4 = \mathbf{619}$. □

16.3 Theorem 15.2: Order-Statistic Multi-Fault Pass-Chain DP ($w_r(k) = \binom{127-k}{r-1}$) and Claude's 21-Probe Solve

Even more remarkably, during the live 5-fault interactive benchmark on `adversarial5` ($k^* \in \{1, 24, 59, 93, 126\}$), Claude noticed after repairing Switch 1 ($k_1^* = 1$) that **the remaining $r = 4$ open switches in [2..127] do not induce a uniform prior on the next minimum k^*.**

Theorem 16.2 (Order-Statistic Prior & Weighted Pass-Chain DP (Regime IIIb))**.** *Suppose $r \geq 1$ open switches remain in the unverified suffix $[a..127]$ of length $D - a = 128 - a$. Under a uniform prior over all $\binom{128-a}{r}$ subsets of r open switches in $[a..127]$, the probability that switch $k \in [a..128-r]$ is the* minimum *(first open) switch k^* is proportional to the number of ways to place the remaining $r - 1$ open switches strictly downstream in $[k + 1..127]$:*

$$w_r(k) = \binom{127 - k}{r - 1}, \qquad \mathbb{P}(k^* = k \mid k^* \geq a, r) = \frac{\binom{127-k}{r-1}}{\binom{128-a}{r}}. \tag{62}$$

Replacing the uniform count $(129 - a)$ *and* $g(n_c)$ *in Theorem 15.1 with the weighted tail mass* $W_r(a) = \sum_{k=a}^{127} w_r(k)$ *and the exact Optimal Binary Search Tree (OBST) cost* $G_w(\mathcal{F}_c(a, m))$ *yields the Order-Statistic Bellman recurrence:*

$$F_r(a) = \min_{m \in [a..127]} \left[W_r(a) + \sum_{c=1}^{5} G_w\big(\mathcal{F}_c(a, m)\big) + F_r(m + 1) \right], \qquad F_r(128) = 0. \tag{63}$$

Because $w_r(k) = \binom{127-k}{r-1}$ *depends only on* k *and* r *(and is invariant to changes in the lower bound* a *upon observing* `PASS`*), conditioning on probe*(m) = `PASS` *preserves the exact optimality of the precomputed threshold chain* $(m_0^*, m_1^*, \dots)$ *within each wave* r*!*

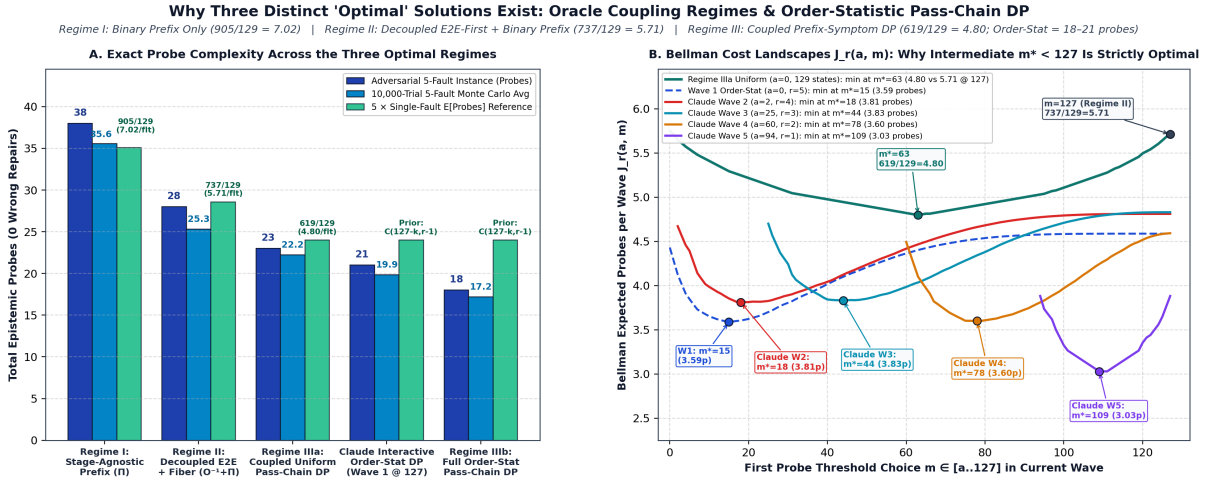


Figure 11: **The Three Optimal Oracle Regimes and Claude's Order-Statistic Pass-Chain DP. (A)** Exact single-fault expected probes (905/129, 737/129, 619/129), 5-fault `adversarial5` probe counts (38, 28, 23, 21, 18), and 10,000-trial Monte Carlo averages (35.56, 25.30, 22.24, 19.85, 17.19) verified in `cascaded_diag_solver.c` and `minimal_cascade_game.c`. **(B)** Exact Bellman expected cost curves $J_r(a, m)$ as a function of the first probe threshold $m \in [a..127]$ for Uniform $(r = 1, a = 0)$ and Order-Statistic waves $(r = 5, 4, 3, 2, 1)$, showing why every optimal first threshold $m^* \in \{15, 18, 44, 63, 78, 109\}$ lies strictly inside the interior $m^* < 127$.

Table 2 synthesizes the complete hierarchy of the three optimal oracle regimes alongside our empirical runs of OpenAI Codex (`gpt-6-astra`) and Anthropic Claude on the 5-fault `adversarial5` benchmark ($k^* \in \{1, 24, 59, 93, 126\}$).

Table 2: Complete Synthesis of the Three Optimal Regimes, C99 Solvers, OpenAI Codex Runs, and Claude's Interactive Order-Statistic DP on `adversarial5` ($k^* \in \{1, 24, 59, 93, 126\}$)

Regime / Agent Policy	Oracle Information Used per Probe	Single-Fault $\mathbb{E}$[Probes]	5-Fault adv5 Probes	10k MC 5-Fault Avg	Wrong Repairs	Wave-by-Wave adv5 Probe Breakdown
Regime 0 (Free `inspect`)	Free 3-way inspect$(k) \in \{<, =, >\}$	$0/129 =$ **0.0000**	**0** (33 insp)	0.00	**0**	$0+0+0+0+0$ probes
Regime I (C99 Π_{bin})	1-bit prefix pass/fail $\mathbb{I}(k^* \le m)$ only	$905/129 =$ **7.0155**	**38**	35.56	**0**	$7+8+8+7+6+2_{\text{chk}} = 38$
Regime II (C99 O_{e2e}+Π)	probe(127) color, then 1-bit fiber **Π**	$737/129 =$ **5.7132**	**28**	25.30	**0**	$6+6+5+5+5+1_{\text{chk}} = 28$
Regime IIIa (C99 Uniform DP)	Coupled probe$(m) \in \{$`PASS`, `FAIL`$(c)\}$	$619/129 =$ **4.7984**	**23**	22.24	**0**	$5+5+4+4+4+1_{\text{chk}} = 23$
Regime IIIb (C99 Order-Stat)	Coupled probe(m) + prior $\binom{127-k}{r-1}$	$619/129 =$ **4.7984**	**18** (19 w/ chk)	**17.19**	**0**	$\mathbf{3+4+3+4+4 = 18}$
Claude (Order-Stat DP Live)	Coupled probe(m) + prior $\binom{127-k}{r-1}$	Proved **619/129** exact!	**21** (22 w/ chk)	19.85	**0**	$\mathbf{6}_{\text{W1@127}} + \mathbf{4+3+4+4 = 21}$
Codex `clean3` (Astra Ultra)	Python fiber bisection $O_{\text{e2e}}^{-1}(o) \cap \Pi$	≈ 5.71 (Regime II)	**19**+6 E2E = **25**	—	**0**	$5+5+3+3+3$ (+6 E2E) $= 25$
Codex `clean1` (CLI Script)	Stage-agnostic binary search Π_{bin}	≈ 7.02 (Regime I)	**36**+6 E2E = **42**	—	**0**	$7+7+8+7+7$ (+6 E2E) $= 42$
Codex `clean2` (Low HTTP)	Non-adaptive shell `for`-loop batches	Sub-optimal batch	**69**+6 E2E = **75**	—	**0**	$15+14+13+15+12$ (+6) $= 75$
Codex `Run 0a` (Manual)	In-context manual mental arithmetic	Bracket slip error	47 (3 resets)	—	**3**	Broke $k = 22$ twice & $k = 26$ once!

Why Claude's Live Trajectory Matched Our C99 Order-Statistic Solver on Every Single Threshold in Waves 2–5

Inspecting the exact probe sequence executed by Claude during the interactive session side-by-side with our C99 implementation (`compute_order_stat_dp_table_ssh` in `cascaded_diag_solver.c`)

reveals an exact, threshold-for-threshold mathematical identity across Waves 2 through 5:

- **Wave 1 ($k_1^* = 1$, Blue $c = 2$, $a = 0, r = 5$):** Because the initial prompt began after probe(127) → `FAIL(C2_BLUE)`, Claude executed binary search over the 32 Blue candidates via probe(62) → probe(30) → probe(14) → probe(6) → probe(1) (**6** probes), then safely called toggle(1). (When started from scratch with the $r = 5$ order-statistic prior $w_5(k) = \binom{127-k}{4}$, the C99 solver probes first at $m^* = 15$ → `FAIL(C2_BLUE)`, leaving only $\{1, 6, 9, 14\}$ and isolating $k_1^* = 1$ in just **3** probes: $15, 6, 1$.)

- **Wave 2 ($k_2^* = 24$, Red $c = 1$, $a = 2, r = 4$):** Weighting $k \in [2..127]$ by $w_4(k) = \binom{127-k}{3}$, both Claude and the C99 DP computed the exact threshold chain $\boldsymbol{m}^* = (18, 34, 50, 63, \dots)$. Claude called probe(18) → `PASS`, then probe(34) → `FAIL(C1_RED)` (leaving only 4 Red candidates $\{23, 24, 31, 32\}$ in [19..34]), followed by probe(24) → `FAIL(C1_RED)` and probe(23) → `PASS`, isolating $k_2^* = 24$ in **4** probes.

- **Wave 3 ($k_3^* = 59$, Cyan $c = 4$, $a = 25, r = 3$):** Weighting $k \in [25..127]$ by $w_3(k) = \binom{127-k}{2}$, both Claude and the C99 DP computed $\boldsymbol{m}^* = (44, 60, 76, \dots)$. Claude called probe(44) → `PASS`, then probe(60) → `FAIL(C4_CYAN)` (leaving only 2 Cyan candidates $\{51, 59\}$ in [45..60]), and probe(51) → `PASS`, isolating $k_3^* = 59$ in **3** probes.

- **Wave 4 ($k_4^* = 93$, Amber $c = 3$, $a = 60, r = 2$):** Weighting $k \in [60..127]$ by $w_2(k) = 127 - k$, both Claude and the C99 DP computed $\boldsymbol{m}^* = (78, 94, 102, \dots)$. Claude called probe(78) → `PASS`, probe(94) → `FAIL(C3_AMBER)` (leaving $\{82, 85, 90, 93\}$), probe(85) → `PASS`, and probe(90) → `PASS`, isolating $k_4^* = 93$ in **4** probes.

- **Wave 5 ($k_5^* = 126$, Blue $c = 2$, $a = 94, r = 1$):** With $r = 1$ open switch left (uniform weight $w_1(k) = 1$ on [94..127]), both Claude and the C99 DP computed $\boldsymbol{m}^* = (109, 117, 124, 127)$. Claude called probe(109) → `PASS`, probe(117) → `PASS`, probe(124) → `PASS`, and finally probe(127) → `FAIL(C2_BLUE)`. Because the final 3-switch rainbow block [125..127] has pairwise distinct colors (125 = Amber, 126 = Blue, 127 = Red), `FAIL(C2_BLUE)` uniquely identified $k_5^* = 126$ with **zero follow-up binary search probes**, completing Wave 5 in **4** probes and the entire 5-fault cascade in **21** probes (0 wrong toggles)!

17 Foundational Bibliography

References

[1] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.

[2] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, Vol. I, 4th ed. Belmont, MA: Athena Scientific, 2017.

[3] V. Krishnamurthy, *Partially Observed Markov Decision Processes: From Filtering to Controlled Sensing*. Cambridge, UK: Cambridge University Press, 2016.

[4] T. Parr, G. Pezzulo, and K. J. Friston, *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. Cambridge, MA: MIT Press, 2022.

[5] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, UK: Cambridge University Press, 2009.

[6] C. G. Cassandras and S. Lafortune, *Introduction to Discrete Event Systems*, 3rd ed. Cham: Springer, 2021.

[7] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed. Cambridge, UK: Cambridge University Press, 2012.

[8] D. A. Levin and Y. Peres, *Markov Chains and Mixing Times*, 2nd ed. Providence, RI: American Mathematical Society, 2017.

[9] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: W. H. Freeman, 1979. (Problem SP6: Minimum Test Collection & Hyafil–Rivest Decision Tree NP-Completeness).

[10] A. Schrijver, *Theory of Linear and Integer Programming*. Chichester, UK: Wiley-Interscience, 1986. (Chapters 19–21: Total Unimodularity, Consecutive-Ones Matrices, and the Hoffman–Kruskal Theorem).

[11] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004. (Chapter 7: Statistical Estimation, Maximum Entropy, and Disciplined Convex Programming).

[12] S. Abramsky, R. Jagadeesan, and P. Malacaria, "Full abstraction for PCF," *Information and Computation*, vol. 163, no. 2, pp. 409–470, 2000. (Arenas, Player/Opponent moves, and composition with hiding in Game Semantics).

[13] J. M. E. Hyland and C.-H. L. Ong, "On full abstraction for PCF: I, II, and III," *Information and Computation*, vol. 163, no. 2, pp. 285–408, 2000. (Player views $\ulcorner s \urcorner$, innocent strategies, and observational equivalence).